

Revision Sheet
Dr. Mahmoud Wahdan

1. What is an algorithm?

- a) A step-by-step procedure for solving a problem**
- b) A programming language
- c) A computer hardware component
- d) An internet protocol

2. What is the purpose of an algorithm?

- a) To display web pages
- b) To store data
- c) To solve a specific problem or perform a task**
- d) To connect to the internet

3. What is the primary purpose of using algorithms in computer science?

- a) To improve internet speed
- b) To process natural language
- c) To solve problems and perform tasks efficiently**
- d) To design computer hardware

4. Which characteristic of an algorithm ensures that it eventually terminates?

- | | |
|-----------------|----------------------|
| a) Definiteness | b) Finiteness |
| c) Correctness | d) Efficiency |

5. Optimization of algorithm means

- a) making that algorithm fast by time and compact by space**
- b) making that algorithm slow by time and large by space
- c) making that algorithm fast by time and large by space
- d) making that algorithm slow by time and compact by space

6. Which characteristic of an algorithm ensures that it produces the desired output for all valid inputs?
- a) Definiteness
 - b) Finiteness
 - c) **Correctness**
 - d) Efficiency
7. What is the primary characteristic of an efficient algorithm?
- a) **It terminates in a reasonable amount of time**
 - b) It produces visually appealing output
 - c) It can handle unlimited amounts of data
 - d) It can solve any type of problem
8. There are two algorithms suppose A takes 1.41 milli seconds while B takes 0.9 milliseconds, which one of them is better considering all other things the same?
- a) A is better than B
 - b) **B is better than A**
 - c) Both are equally good
 - d) None of the mentioned
9. Which type of algorithm arranges elements in a specific order, such as numerical or alphabetical?
- a) Searching Algorithms
 - b) **Sorting Algorithms**
 - c) Graph Algorithms
 - d) Dynamic Programming Algorithms
10. Which type of algorithm finds a specific element or value in a collection of data?
- a) Sorting Algorithms
 - b) Graph Algorithms
 - c) Dynamic Programming Algorithms
 - d) **Searching Algorithms**

11. Which type of algorithm involves exploring all possible solutions to a problem by systematically trying different options?

- a) **Brute-Force Algorithm**
- b) Backtracking Algorithm
- c) Greedy Algorithm
- d) Divide and Conquer Algorithm

12. Which algorithmic approach involves breaking a problem into smaller, more manageable sub-problems and combining their solutions to solve the main problem?

- a) Greedy Algorithm
- b) Backtracking Algorithm
- c) Dynamic Programming Algorithm
- d) **Divide and Conquer Algorithm**

13. What is the time, and space complexity of the following code:

```
int a = 0, b = 0;
for (i = 0; i < N; i++) {
    a = a + rand();
}
for (j = 0; j < M; j++) {
    b = b + rand();
}
```

- a) $O(N * M)$ time, $O(1)$ space
- b) $O(N + M)$ time, $O(N + M)$ space
- c) **$O(N + M)$ time, $O(1)$ space**
- d) $O(N * M)$ time, $O(N + M)$ space

Explanation: The first loop is $O(N)$ and the second loop is $O(M)$. Since N and M are independent variables, so we can't say which one is the leading term. Therefore **Time complexity** of the given problem will be $O(N+M)$.

Since variables size does not depend on the size of the input, therefore **Space Complexity** will be constant or $O(1)$

14. What is the time complexity of the following code:

```
int a = 0;
for (i = 0; i < N; i++) {
    for (j = N; j > i; j--) {
        a = a + i + j;
    }
}
```

- a) $O(N)$
- b) $O(N \cdot \log(N))$
- c) $O(N \cdot \text{Sqrt}(N))$
- d) **$O(N \cdot N)$**

Explanation:

The above code runs total no of times
= $N + (N - 1) + (N - 2) + \dots 1 + 0$
= $N * (N + 1) / 2$
= $1/2 * N^2 + 1/2 * N$
 $O(N^2)$ times.

15. What is the time complexity of the following code?

```
int a = 0, i = N;
while (i > 0) {
    a += i;
    i /= 2;
}
```

- a) $O(N)$
- b) $O(\text{Sqrt}(N))$
- c) $O(N/2)$
- d) **$O(\log N)$**

16. What is the time complexity of the following code?

```
int i, j, k = 0;
for (i = n / 2; i <= n; i++) {
    for (j = 2; j <= n; j = j * 2) {
        k = k + n / 2;
    }
}
```

- a) $O(n)$
- b) $O(n \log n)$**
- c) $O(n^2)$
- d) $O(n^2 \log n)$

Explanation: If you notice, j keeps doubling till it is less than or equal to n . Several times, we can double a number till it is less than n would be $\log(n)$.

Let's take the examples here.

for $n = 16$, $j = 2, 4, 8, 16$

for $n = 32$, $j = 2, 4, 8, 16, 32$

So, j would run for $O(\log n)$ steps.

i runs for $n/2$ steps.

So, total steps = $O(n/2 * \log(n)) = O(n \log n)$

17. Algorithm A and B have a worst-case running time of $O(n)$ and $O(\log n)$, respectively. Therefore, algorithm B always runs faster than algorithm A.

- a) True
- b) False**

Explanation:

The Big-O notation provides an asymptotic comparison in the running time of algorithms. For $n < n^0$, algorithm A might run faster than algorithm B, for instance.

18. Which of the following best describes the useful criterion for comparing the efficiency of algorithms?

- a) Time
- b) Memory
- c) Both of the above**
- d) None of the above

19. What is brute force?

- a) Solving a problem using a systematic approach that optimizes time and resources. Best for structured problems.
- b) Solving a problem by breaking it into smaller, more manageable tasks. Suitable for complex problems.
- c) Solving a problem directly but is often very inefficient. Can be used on a variety of problems.**
- d) Solving a problem using advanced mathematical techniques. Efficient for specific problems only.

20. Brute Force strengths and weaknesses?

- a) Strengths: universal, simple, reasonable algorithms**
Weakness: Inefficient, slow, unconstructive.
- b) A brute force search algorithm that searches each element in list until it finds a match to a key.
- c) Search that starts at a specified vertex and explores the Neighbours nodes first. Before moving to the next level of Neighbours.
- d) Problems that require you to search through all possible combinations to find the best one

21. A sorting technique is called stable if

- a) If it takes $O(n \log n)$ time.
- b) It uses divide and conquer technique.
- c) Relative order of occurrence of non-distinct elements is maintained.**
- d) It takes $O(n)$ space.

Explanation

A stable sort keeps equal elements in the same order they appeared in the input.

22. Which of the following sorting algorithms repeatedly compares adjacent elements and swaps them if they are in the wrong order?

- a) Merge Sort
- b) Bubble Sort**
- c) Quick Sort
- d) Selection Sort

23. You are running a library catalog. You know that the books in your collection are almost in sorted ascending order by title, with the exception of one book which is in the wrong place. You want the catalog to be completely sorted in ascending order. Which sorting algorithm would be the best (i.e., most efficient)?

- a) Insertion Sort**
- b) Merge Sort
- c) Radix Sort
- d) Heap Sort

24. Which sorting algorithm will take least time when all elements of input array are identical? Consider typical implementations of sorting algorithms.

- a) **Insertion Sort**
- b) Heap sort
- c) Merge sort
- d) Selection sort

Explanation

The insertion sort will $O(n)$ time when input array is already sorted.

25. Which of the following sorting algorithms in its typical implementation gives best performance when applied on an array which is sorted or almost sorted (maximum 1 or two elements are misplaced).

- a) Quick sort
- b) Heap sort
- c) Merge sort
- d) Insertion sort**

Explanation

Insertion sort takes linear time when input array is sorted or almost sorted (maximum 1 or 2 elements are misplaced). All other sorting algorithms mentioned above will take more than linear time in their typical implementation.

26. Consider the following array.

23	32	45	69	72	73	89	97
----	----	----	----	----	----	----	----

Which algorithm out of the following options uses the least number of comparisons (among the array elements) to sort the above array in ascending order?

- a) Selection sort
- b) Mergesort
- c) Insertion sort**
- d) Quicksort using the last element as pivot

Explanation

Since, given array is almost sorted in ascending order, so [Insertion sort](#) will give its best case with time complexity of order $O(n)$.

27. Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

- a) Merge Sort
- b) Selection Sort**
- c) Heap Sort
- d) Insertion Sort

Explanation

Selection sort makes $O(n)$ swaps which is the minimum among all sorting algorithms mentioned above.

28. Assume that we use Bubble Sort to sort n distinct elements in ascending order. When does the best case of Bubble Sort occur?

- a) When elements are sorted in ascending order**
- b) When elements are sorted in descending order
- c) When elements are not sorted by any order
- d) There is no best case for Bubble Sort. It always takes $O(n*n)$ time

Explanation

In the best-case scenario, the array is **already sorted**, but just in case, bubble sort performs $O(n)$ comparisons. As a result, the time complexity of bubble sort in the best-case scenario is $O(n)$.

29. What is a “binary search algorithm”?

- a) An algorithm that finds the position of a target value in a sorted array**
- b) An algorithm that searches unsorted data
- c) An algorithm that sorts data
- d) An algorithm that searches for multiple values

30. What is a “merge sort algorithm”?

- a) **An algorithm that divides the data into smaller sublists and merges them back together**
- b) An algorithm that sorts data in reverse order
- c) An algorithm that selects the smallest value
- d) An algorithm that finds duplicates

31. Which of the following statements is correct with respect to insertion sort?

*Online - can sort a list at runtime.

*Stable - doesn't change the relative order of elements with equal keys.

- a) **Insertion sort is stable, online but not suited well for large number of elements.**
- b) Insertion sort is unstable and online
- c) Insertion sort is online and can be applied to more than 100 elements
- d) Insertion sort is stable & online and can be applied to more than 100 elements

Explanation

Time taken by algorithm is good for small number of elements, but increases quadratically for large number of elements.

32. Which is the correct order of the following algorithms with respect to their time Complexity in the best case?

- a) Merge sort > Quick sort > Insertion sort > selection sort
- b) **insertion sort < Quick sort < Merge sort < selection sort**
- c) Merge sort > selection sort > quick sort > insertion sort
- d) Merge sort > Quick sort > selection sort > insertion sort

Explanation

In best case,

Quick sort: $O(n \log n)$

Merge sort: $O(n \log n)$

Insertion sort: $O(n)$

Selection sort: $O(n^2)$

33. What is a “selection sort algorithm”?

- a) **An algorithm that repeatedly selects the smallest element and places it in order**
- b) An algorithm that randomly selects elements
- c) An algorithm that sorts data in reverse order
- d) An algorithm that only sorts two elements

34. What is selection sort?

- a) **A brute force sorting algorithm that scans array to find smallest element to swap with first element over a series of $n-1$ passes until its sorted. $O(n^2)$**
- b) A brute force sorting algorithm that compares adjacent elements and swapping them up which pushes largest element up until $n-1$ passes. $O(n^2)$
- c) A brute force algorithm that searches for a pattern of characters within a text. Worst Case $(n-m+1)$
- d) A brute force search algorithm that searches each element in list until it finds a match to a key.

35. What is a “bubble sort algorithm”?

- a) **An algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order**
- b) An algorithm that selects the largest element
- c) An algorithm that sorts data in reverse order
- d) An algorithm that only sorts two elements

36. What is a bubble sort?

- a) An efficient sorting algorithm that divides elements and merges them. $O(n \log n)$
- b) A brute force sorting algorithm that compares adjacent elements and swapping them up which pushes large element up until $n-1$ passes. $O(n^2)$**
- c) A sorting algorithm that sorts elements by partitioning and conquering. $O(n)$
- d) A sorting algorithm that uses recursion to sort elements. $O(\log n)$

37. Which of the following is an example of a searching algorithm?

- a) Quick Sort
- b) Binary Search**
- c) Merge Sort
- d) Bubble Sort

38. Binary insertion sorting (insertion sort that uses binary search to find each insertion point) requires $O(n \log n)$ total operations.

- a) True
- b) False**

Explanation

While binary insertion sorting improves the time it takes to find the right position for the next element being inserted, it may still take $O(n)$ time to perform the swaps necessary to shift it into place. This results in an $O(n^2)$ running time, the same as that of insertion sort.

39. Consider an array of elements $\text{arr}[5] = \{5,4,3,2,1\}$, what are the steps of insertions done while doing insertion sort in the array.

a) 4 5 3 2 1

3 4 5 2 1

2 3 4 5 1

1 2 3 4 5

b) 5 4 3 1 2

5 4 1 2 3

5 1 2 3 4

1 2 3 4 5

c) 4 3 2 1 5

3 2 1 5 4

2 1 5 4 3

1 5 4 3 2

d) 4 5 3 2 1

2 3 4 5 1

3 4 5 2 1

1 2 3 4 5

Explanation

In the insertion sort, just imagine that the first element is already sorted and all the right-side Elements are unsorted, we need to insert all elements one by one from left to right in the sorted Array.

Sorted: 5

unsorted: 4 3 2 1

Insert all elements less than 5 on the left (Considering 5 as the key).

Now key value is 4 and array will look like this

Sorted: 4 5

unsorted: 3 2 1

Similarly for all the cases the key will always be the newly inserted value and all the values will be compared to that key and inserted in to proper position.

40. What is sequential search?

- a) A search algorithm that sorts the list before searching.
- b) A randomized search algorithm that checks elements randomly.
- c) A brute force search algorithm that searches each element in list until it finds a match to a key.**
- d) An efficient search algorithm that uses binary search to find a key.

41. A brute force search algorithm that searches each element in list until it finds a match to a key.

- a) What is a Breadth first search?
- b) What is an exhaustive search?
- c) What is brute force?
- d) What is sequential search?**

42. Which type of algorithm finds a specific element or value in a collection of data?

- a) Sorting Algorithms
- b) Graph Algorithms
- c) Dynamic Programming Algorithms
- d) Searching Algorithms**

43. The necessary condition for using binary search in an array is

- a) The array should not be too long
- b) The array should of more size
- c) The array should be sorted**
- d) None of these

Explanation

To apply binary search on an array, the array should be sorted. Binary Search is defined as a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log N)$.

44. What is the time complexity of the linear search algorithm for searching an array of size n ?

- a) $O(1)$
- b) $O(n)$**
- c) $O(\log n)$
- d) $O(n^2)$

45. What is the time complexity of a binary search method on a sorted list of length n ?

- a) $O(1)$
- b) $O(n)$
- c) $O(\log n)$**
- d) $O(n^2)$

46. What is string matching?

- a) An algorithm that compresses a text to find a pattern.
- b) A technique for summarizing key characters in a text.
- c) A brute force algorithm that searches for a pattern of characters within a text. Worst Case $(n-m+1)$**
- d) A method for encrypting characters within a text.

47. What is the time complexity of sorting an array using Bubble Sort?

- a) $O(n)$
- b) $O(n^3)$
- c) $O(n \log_2 n)$
- d) $O(n^2)$**

48. Which approach is based on computing the distance between each pair of distinct points and finding a pair with the smallest distance?

a) Brute force

b) Exhaustive search

c) Divide and conquer

d) Branch and bound

Explanation:

Brute force is a straight forward approach that solves closest pair problem using that algorithm.

49. What is the runtime efficiency of using brute force technique for the closest pair problem?

a) $O(N)$

b) $O(N \log N)$

c) $O(N^2)$

d) $O(N^3 \log N)$

Explanation:

The efficiency of closest pair algorithm by brute force technique is mathematically found to be $O(N^2)$.

50. What is the basic operation of closest pair algorithm using brute force technique?

a) Euclidean distance

b) Radius

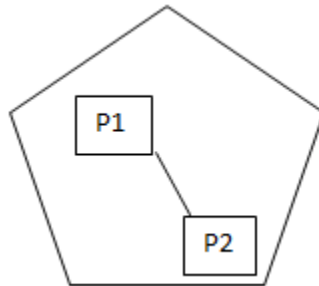
c) Area

d) Manhattan distance

Explanation:

The basic operation of closest pair algorithm is Euclidean distance and its formula is given by $d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

51. Which of the following strategies does the following diagram depict?



- a) Divide and conquer strategy
- b) Brute force**
- c) Exhaustive search
- d) Backtracking

Explanation:

Brute force is a straight forward approach to solve critical problems. Here, we use brute force technique to find the closest distance between p1 and p2.

52. Exhaustive searching is a strategy used for solving:

- a) Simple linear problems
- b) Analytical problems
- c) Combinatorial problems**
- d) Optimization problems

53. Which of the following is best search algorithm for combinatorial problem?

- a) Linear Search
- b) Binary Search
- c) Ternary Search
- d) Exhaustive Search**

54. Problems that require you to search through all possible combinations to find the best one

a) What is selection sort?

b) What is an exhaustive search?

c) What are properties of a depth first search?

d) What is sequential search?

55. Consider a job scheduling problem with 4 jobs J_1, J_2, J_3, J_4 and with corresponding deadlines: $(d_1, d_2, d_3, d_4) = (4, 2, 4, 2)$. Which of the following is **not** a feasible schedule without violating any job schedule?

a) J_2, J_4, J_1, J_3

b) J_4, J_1, J_2, J_3

c) J_4, J_2, J_1, J_3

d) J_4, J_2, J_3, J_1

Explanation

Option A: 2,2,4,4. No problem.

Option B: 2,4,2,4.

The third job in this sequence had to be completed under 2 time units. But it took 3 time units. **This isn't an optimal solution**, hence the answer.

Option C: 2,2,4,4. No problem.

Option D: 2,2,4,4. No problem.

56. Which type of knapsack problem allows loading a knapsack with fractions of items?

a) Fractional knapsack problem

b) Integer knapsack problem

c) Unbounded knapsack problem

d) Bounded knapsack problem

57. What is the profit for the following table using Knapsack Problem

Total Capacity of Sack?	10 Units	
Stock	Quantity	Profit
Item 1	2	20
Item 2	4	30
Item 3	7	25
Item 4	3	20
Item 5	1	10

- a) 70 b) 100 c) 105 **d) 80**

58. What is the basic difference between the Knapsack Problem and Container Loading Problem?

- a) Container Loading Problem and Knapsack Problem are constrained by the total capacity
- b) Container Loading Problem deals with maximum Profit whereas the Knapsack problem deals with maximum capacity
- c) Container Loading Problem deals with maximum capacity whereas the Knapsack deals with the capacity and profit**
- d) Container Loading Problem has constraints involves two independent parameters whereas the Knapsack Problem has constraints involves three different parameters.

59. What is the objective of the Knapsack problem?

- a) To get maximum total value in the Knapsack.**
- b) To get minimum total in the Knapsack.
- c) To get the maximum weight in the Knapsack.
- d) To get the minimum weight in the Knapsack.

60. Efficient Method for solving Assignment Problem is

- a) **Hungarian Method**
- b) numeration Method
- c) Simplex Method
- d) Transportation Method

61. What is an exhaustive search?

- a) Problems that need only a partial search for a solution
- b) Problems that involve searching for the shortest path
- c) Problems that require a single step to find the solution
- d) **Problems that require you to search through all possible combinations to find the best one**

62. What is a depth first search?

- a) **Search that starts at a specified vertex and explores as far as possible along each branch before back tracking and continuing the search until there are no more paths. It will then go to an unvisited vertex and start again.**
- b) A brute force sorting algorithm that scans array to find smallest element to swap with first element over a series of $n-1$ passes until its sorted. $O(n^2)$
- c) Problems that require you to search through all possible combinations to find the best one
- d) Search that starts at a specified vertex and explores the Neighbours nosed first. Before moving to the next level of Neighbours.

63. What is the primary difference between BFS and DFS?

- a) BFS uses a stack, while DFS uses a queue
- b) **BFS explores all neighbors first, while DFS goes deep before backtracking**
- c) BFS is for weighted graphs, DFS for unweighted
- d) DFS is faster than BFS

Explanation:

BFS uses a queue to explore all neighbors level by level, while DFS uses a stack or recursion to explore as far as possible along each branch.

64. What are properties of a depth first search?

- a) Search that starts at a specified vertex and explores as far as possible along each branch before back tracking and continuing the search until there are no more paths. It will then go to an unvisited vertex and start again.
- b) Uses a queue, one ordering, tree and cross edges, used to check connectivity, acyclicity, and min-edge paths, Runtime for Adj matrix $O(V^2)$. Runtime for adj list is $O(V+E)$.
- c) Uses a stack, two orderings (along tree and back edges), used to check connectivity, acyclicity, and articulation points, Runtime for Adj matrix $O(V^2)$. Runtime for adj list is $O(V+E)$.**
- d) Search that starts at a specified vertex and explores the Neighbours nosed first. Before moving to the next level of Neighbours.

65. Search that starts at a specified vertex and explores as far as possible along each branch before back tracking and continuing the search until there are no more paths. It will then go to an unvisited vertex and start again.

- a) What is a depth first search?**
- b) What is brute force?
- c) What is sequential search?
- d) What is a bubble sort?

66. Uses a queue, one ordering, tree and cross edges, used to check connectivity, acyclicity, and min-edge paths, Runtime for Adj matrix $O(V^2)$. Runtime for adj list is $O(V+E)$.

a) What is brute force?

b) What is selection sort?

c) What are properties of a breadth first search?

d) What is string matching?

67. Uses a stack, two orderings (along tree and back edges), used to check connectivity, acyclicity, and articulation points, Runtime for Adj matrix $O(V^2)$. Runtime for adj list is $O(V+E)$.

a) What are properties of a depth first search?

b) What is brute force?

c) What is a bubble sort?

d) Brute Force strengths and weaknesses?

68. What is a Breadth first search?

a) Search that starts at a specified vertex and explores the Neighbours nosed first. Before moving to the next level of Neighbours.

b) Search that prioritizes the most distant Neighbours before closer ones.

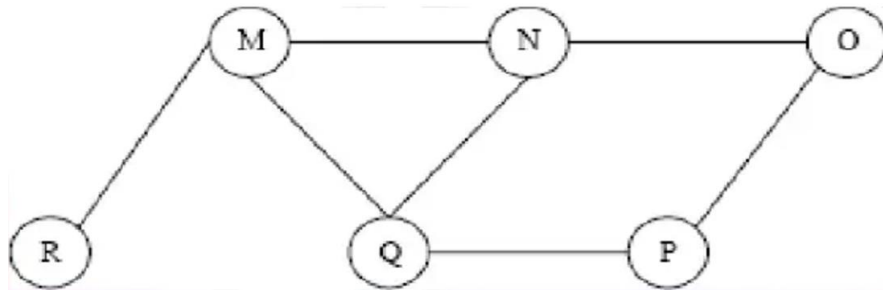
c) Search that explores the deepest branch first before moving to Neighbours.

d) Search that starts at a random vertex and explores all paths simultaneously.

69. What is common in three types of traversals (in order, preorder and postorder)?

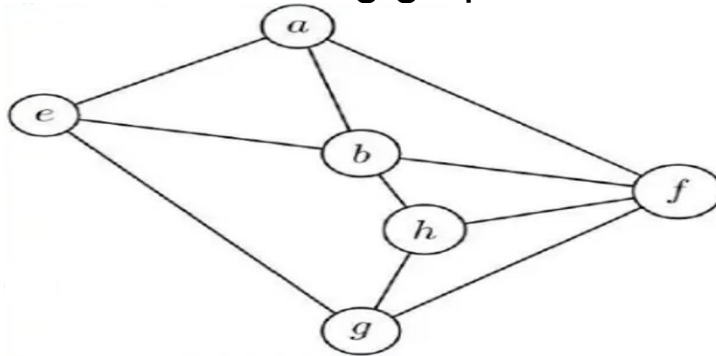
- a) Root is visited before right subtree
- b) Left subtree is always visited before right subtree**
- c) Root is visited after left tree
- d) All of the above

70. The BFS algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is:



- a) MNOPQR
- b) NQMPOR
- c) QMNPOR**
- d) QMNPOR

71. Consider the following graph:



Among the following sequences:

- i. a b e g h f
- ii. a b f e h g
- iii. a b f h g e
- iv. a f g h b e

Which are depth first traversals of the above graph?

- a) I, II and IV only
- b) I and IV only
- c) II, III and IV only
- d) I, III, and IV only**

72. Which algorithmic approach involves breaking a problem into smaller, more manageable sub-problems and combining their solutions to solve the main problem?

- a) Greedy Algorithm
- b) Backtracking Algorithm
- c) Dynamic Programming Algorithm
- d) Divide and Conquer Algorithm**

73. Which of the following algorithms is NOT a divide & conquer algorithm by nature?

- a) Euclidean algorithm to compute the greatest common divisor
- b) Heap Sort**
- c) Cooley-Tukey fast Fourier transform
- d) Quick Sort

74. Which of the following is NOT a divide and conquer method?

- a) Binary Search
- b) Merge Sort
- c) Quick Sort
- d) Heap Sort**

75. What is the very basic principle of the Divide and Conquer algorithm

- a) Problem-solving through subdivision
- b) Solution of the Problems through One Recursive Function**
- c) Solving problems without recursion
- d) Sequential problem solving

Explanation:

Divide and Conquer involves decomposing a given problem into smaller subproblems, resolving them recursively, and then combining their solutions.

76. Within the Divide and Conquer paradigm, the "Conquer" step entails:

- a) Merging the solutions of the subproblems to provide a good solution for the original problem.
- b) Divide the problem into many component subproblems**
- c) The base case of the recursion is implemented
- d) None of the above

Explanation:

In the "Conquer" step, subproblem solutions are merged to solve the original problem.

77. What is the key idea of the Divide and Conquer paradigm?

- a) Divide and solve recursively only
- b) Repeat iteration until convergence
- c) Merge all parts together without division
- d) Divide the problem, solve independently, and combine results**

Explanation

The Divide and Conquer paradigm involves breaking a problem into smaller subproblems, solving them independently, and then combining the results to solve the original problem.

78. Which of the following is NOT a typical step in Divide and Conquer algorithms?

- a) Conquer
- b) Divide
- c) Memorize**
- d) Combine

Explanation

The typical steps in Divide and Conquer are to divide the problem, conquer the subproblems, and combine the results. Memorization is not a standard step in this paradigm.

79. Maximum Subarray Sum problem is to find the subarray with maximum sum. For example, given an array {12, -13, -5, 25, -20, 30, 10}, the maximum subarray sum is 45. The naive solution for this problem is to calculate sum of all subarrays starting with every element and return the maximum of all. We can solve this using Divide and Conquer, what will be the worst-case time complexity using Divide and Conquer?

- a) $O(n)$
- b) $O(n \log n)$**
- c) $O(\log n)$
- d) $O(n^2)$

80. Which of the following sorting technique is an example of divide & conquer technique?

- a) Selection sort
- b) Quick sort**
- c) Bubble sort
- d) Insertion sort

81. Which of the algorithms uses the Divide and Conquer approach?

- a) Bubble Sort
- b) Linear Search
- c) Merge Sort**
- d) Insertion Sort

82. Which problem is NOT typically solved using Divide and Conquer?

- a) Quick Sort
- b) Linear Search**
- c) Merge Sort
- d) Binary Search

Explanation

Linear Search is a sequential search algorithm that does not utilize the Divide and Conquer strategy, unlike Binary Search, Merge Sort, and Quick Sort.

83. The Master Theorem helps in:

- a) Dynamic programming optimization
- b) Finding recursion base case
- c) Solving divide and conquer recurrence relations**
- d) Solving linear equations

Explanation

The Master Theorem is a method for analyzing the time complexity of divide and conquer algorithms by solving their recurrence relations.

84. Which sorting algorithm does not use Divide and Conquer?

- | | |
|-----------------------|------------------|
| a) Bubble Sort | b) Binary Search |
| c) Merge Sort | d) Quick Sort |

Explanation

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, comparing adjacent elements, and does not divide the problem into smaller subproblems.

85. What is the number of comparisons in binary search for 16 elements?

- a) 4 b) 8 c) 16 **d) 5**

Explanation

In binary search, the number of comparisons is logarithmic in relation to the number of elements. For 16 elements, it takes 5 comparisons, as $\log_2(16) = 4$.

86. The Closest Pair of Points problem can be efficiently implemented through the Divide and Conquer approach as follows:

- a) $O(n)$
b) $O(n \log n)$
c) $O(n^2)$
d) $O(\log n)$

Explanation:

The Closest Pair of Points problem can be spread over $O(n \log n)$ time via the Divide and Conquer strategy.

87. The algorithms for the following are often good examples of the Divide and Conquer paradigm.

- a) Sorting
b) Searching
c) Both Sorting and Searching
d) Graph Traversal

Explanation:

Algorithms for sorting and searching employ Divide and Conquer increasingly.

88. The divide-and-conquer paradigm is suitable for problems that are inherently:

- a) Large in size
- b) Parallel in nature**
- c) Non-recursive in nature
- d) Difficult to divide

89. Which of the following algorithm design technique is used in merge sort?

- a) Greedy method
- b) Backtracking
- c) Dynamic programming
- d) Divide and Conquer**

Explanation

[Merge sort](#) uses **Divide and Conquer** approach to sort the elements. Array is recursively divided in two halves till the size becomes 1. Once the size of array becomes 1, the merge process comes into action and starts merging the arrays back till the complete array is merged.

90. What principle is merge sort based on?

- a) Greedy
- b) Backtracking
- c) Dynamic Programming
- d) Divide and Conquer**

Explanation

Merge sort is a divide and conquer algorithm, which means it divides the problem into smaller subproblems, solves them independently, and then combines the results.

91. What is the time complexity of merge sort?

- a) $O(\log n)$
- b) $O(1)$
- c) $O(n \log n)$**
- d) $O(n)$

Explanation

The time complexity of merge sort is $O(n \log n)$, which reflects the division of the array and the merging process that takes linear time.

92. When does merge sort perform poorly?

- a) When memory is limited**
- b) Sorting
- c) Linked lists
- d) On large arrays

Explanation

Merge sort performs poorly when memory is limited because it requires additional space proportional to the size of the input array for merging.

93. Within the Merge Sort algorithm, what is the objective of merging?

- a) Dividing the given array into smaller subarrays
- b) Merging two sorted halves to make a single bottom-up merge is the process.**
- c) Choosing a pivot element
- d) Recursively sorting subarrays

Explanation:

In Merge Sort, the "merge" step involves merging two sorted halves and creating one unique, sort array for both.

94. A list of n strings, each of length n , is sorted into lexicographic order using the merge-sort algorithm. The worst-case running time of this computation is

- a) $(n \cdot \log(n))$
- b) $(n^2 \log n)$**
- c) $(n^2 + \log(n))$
- d) (n^2)

Explanation

The recurrence tree for merge sort will have height $\log(n)$. And $O(n^2)$ work will be done at each level of the recurrence tree (Each level involves n comparisons and comparison takes $O(n)$ time in the worst case).

So, time complexity of this Merge Sort will be $O(n^2 \log n)$

95. Which of the following sorting algorithms has the lowest worst-case complexity?

- a) Merge Sort**
- b) Bubble Sort
- c) Quick Sort
- d) Selection Sort

Explanation

Worst-case complexities for the above sorting algorithms are as follows:

- **Merge Sort: $O(n \cdot \log(n))$**
- **Bubble Sort: $O(n^2)$**
- **Quick Sort: $O(n^2)$**
- **Selection Sort: $O(n^2)$**

96. You have to sort 1 GB of data with only 100 MB of available main memory. Which sorting technique will be most appropriate?

- a) Heap sort
- b) Merge sort**
- c) Quick sort
- d) Insertion sort

Explanation

The data can be sorted using external sorting which uses merging technique. This can be done as follows:

- 1. Divide the data into 10 groups each of size 100.**
- 2. Sort each group and write them to disk.**
- 3. Load 10 items from each group into main memory.**
- 4. Output the smallest item from the main memory to disk. Load the next item from the group whose item was chosen.**
- 5. Loop step #4 until all items are not outputted.**

The step 3-5 is called as merging technique.

97. Which of the following is true about merge sort?

- a) Merge Sort works better than quick sort if data is accessed from slow sequential memory.
- b) Merge Sort is stable sort by nature
- c) Merge sort outperforms heap sort in most of the practical situations.
- d) All of the above.**

Explanation

Merge Sort satisfies all the three options given above.

98. Given two sorted list of size m and n respectively. The number of comparisons needed the worst case by the merge sort algorithm will be

- a) $m \times n$
- b) maximum of m and n
- c) minimum of m and n
- d) $m + n - 1$**

Explanation

To merge two lists of size m and n , we need to do $m+n-1$ comparisons in worst case. Since we need to merge 2 at a time, the optimal strategy would be to take smallest size lists first. The reason for picking smallest two items is to carry minimum items for repetition in merging.

99. What is the primary purpose of the Quick Sort algorithm?

- a) To sort data**
- b) To search data
- c) To merge data
- d) To delete data

100. What is the average time complexity of the Quick Sort algorithm?

- a) $O(n)$
- b) $O(n \log n)$**
- c) $O(n^2)$
- d) $O(\log n)$

101. Which of the following is a key characteristic of the Quick Sort algorithm?

- a) It is a stable sort
- b) It uses divide and conquer**
- c) It is a linear sort
- d) It sorts in place

Explanation

Quick Sort utilizes the divide and conquer strategy to sort elements.

102. What is the Divide and Conquer sorting algorithm, which sorts its elements in place?

- a) Merge Sort
- b) QuickSort**
- c) Bubble Sort
- d) Insertion Sort

Explanation:

QuickSort belongs among the in-place sorting algorithms that follow the principle of Divide and Conquer.

103. In Quick Sort, what is the role of the pivot element?

- a) To split the array**
- b) To store results
- c) To count iterations
- d) To merge sorted arrays

Explanation

The pivot element is used to divide the array into sub-arrays.

104. In the Quicksort algorithm, how is the array divided in the first stage?

- a) Equally into two parts
- b) Using a pivot element**
- c) Randomly
- d) Into three subarrays

105. Which of the following best describes the worst-case time complexity of Quick Sort?

- a) $O(n \log n)$
- b) $O(n)$
- c) $O(1)$
- d) $O(n^2)$**

Explanation

The worst-case time complexity occurs when the smallest or largest element is always chosen as the pivot, leading to $O(n^2)$.

106. Which of the following is not a stable sorting algorithm in its typical implementation.

- a) Insertion Sort
- b) Merge Sort
- c) Quick Sort**
- d) Bubble Sort

Explanation

Quick Sort is not a stable sorting algorithm in its typical implementation.

107. Suppose we are sorting an array of eight integers using quicksort, and we have just finished the first partitioning with the array looking like this:

2 5 1 7 9 12 11 10

Which statement is correct?

a) The pivot could be either the 7 or the 9.

b) The pivot could be the 7, but it is not the 9

c) The pivot is not the 7, but it could be the 9

d) Neither the 7 nor the 9 is the pivot.

Explanation

7 and 9 both are at their correct positions (as in a sorted array). Also, all elements on left of 7 and 9 are smaller than 7 and 9 respectively and on right are greater than 7 and 9 respectively.

108. Which of the following sorting algorithms has the minimum running time complexity in the best and average case?

a) Insertion sort, Quick sort

b) Quick sort, Quick sort

c) Quick sort, Insertion sort

d) Insertion sort, Insertion sort

Explanation

Insertion sort has a best-case complexity of $O(n)$, if the array is already sorted while it has an average case complexity of $O(n^2)$

Quick sort has a best-case complexity of $O(n \log n)$, while it has an average case complexity of $O(n \log n)$ also.

109. What is the primary strategy that makes quicksort a divide-and-conquer algorithm?

- a) **Partitioning the array around a pivot, then recursively sorting sub-arrays**
- b) Dividing the array in half and merging sorted portions
- c) Repeatedly swapping adjacent elements until sorted
- d) Selecting the smallest element and moving it to the front

Explanation

Quicksort employs a divide-and-conquer strategy by selecting a pivot element and partitioning the array into two sub-arrays: elements less than the pivot and those greater. It then recursively sorts these sub-arrays, effectively breaking down the sorting task into smaller, manageable parts, leading to efficient overall sorting.

110. Visiting each element in a tree is known as:

- a) Inserting
- b) Merging
- c) Sorting
- d) **Traversing**

111. The following three are known to be the preorder, inorder and postorder sequences of a binary tree. But it is not known which is which.

- I. MBCAFHPYK
- II. KAMCBYPFH
- III. MABCKYFPH

Pick the true statement from the following.

- a) I and II are preorder and inorder sequences, respectively
- b) I and III are preorder and postorder sequences, respectively
- c) II is the inorder sequence, but nothing more can be said about the other two sequences
- d) **II and III are the preorder and inorder sequences, respectively**

Explanation

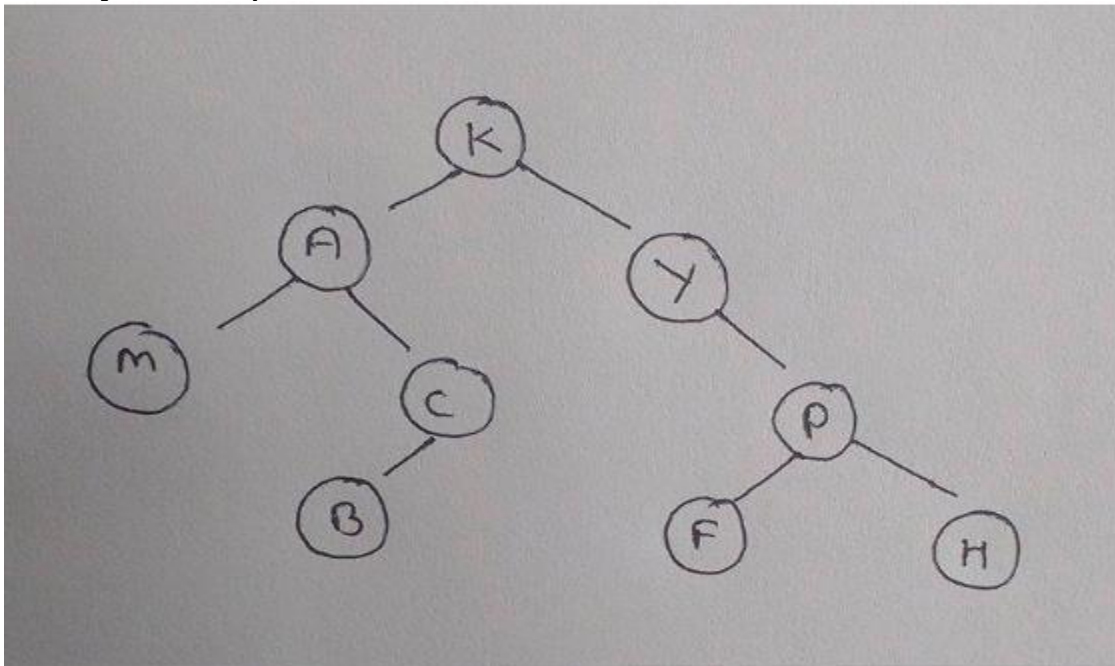
The approach to solve this question is to first find 2 sequences whose first and last element is same. The reason being first element in the Pre-order of any binary tree is the root and last element in the Post-order of any binary tree is the root. Looking at the sequences given,

Pre-order = KAMCBYPFH

Post-order = MBCAFHPYK

Left-over sequence MABCKYFPH will be in order.

Since we have all the traversals identified, let's try to draw the binary tree if possible.



112. The in-order and pre-order traversal of a binary tree are d b e a f c g and a b d e c f g respectively. The post order traversal of a binary tree is

- a) e d b g f c a
- b) e d b f g c a
- c) d e b f g c a**
- d) d e f g b c a

113. Which of the following sequences denotes the post order traversal sequence of the given tree?

a

/ \

b e

/ \ /

c d f

/

g

a) f e g c d b a

b) g c b d a f e

c) g c d b f e a

d) f e d g c b a

114. A complete n-ary tree is one in which every node has 0 or n sons. If x is the number of internal nodes of a complete n-ary tree, the number of leaves in it is given by

a) $x(n-1)+1$

b) $xn-1$

c) $xn+1$

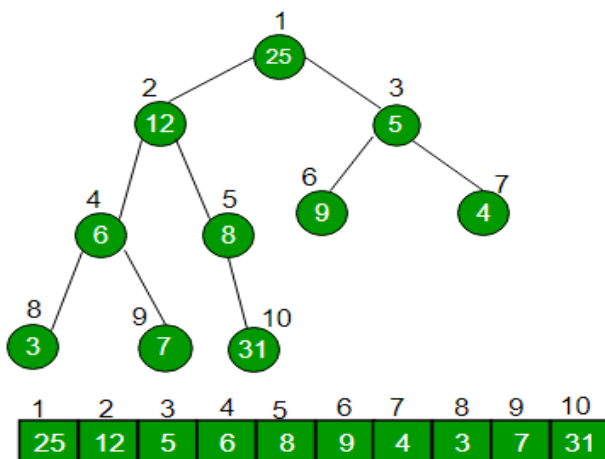
d) $x(n+1)$

115. Which of the following statement is false?

- a) A tree with n nodes has $(n-1)$ edges.
- b) A labeled rooted binary tree can be uniquely constructed given its postorder and preorder traversal results.**
- c) A complete binary tree with n internal nodes has $(n+1)$ leaves.
- d) The maximum number of nodes in a binary tree of height h is $(2^{(h+1)} - 1)$.

Explanation

Both option (A) and (D) are correct. But option (B) is false because "a labeled rooted binary tree **can not be uniquely** constructed given using its postorder and preorder traversal results". It needs inorder and preorder/postorder for uniquely constructed binary tree. Option (C) is also false, take counter example in given tree



Therefore, both option (B) and (C) are false.

116. Which of following option is not correct regarding depth first searching?

- a) In a depth-first traversal of a graph G with V vertices, E edges are marked as tree edges. The number of connected components in G is $(E - V)$.
- b) Depth-first search requires $O(V^2)$ time if implemented with an adjacency matrix.
- c) Depth-first search requires $O(V + E)$ time if implemented with adjacency lists
- d) None of these

Explanation

In a depth-first traversal of a graph G with V vertices, E edges are marked as tree edges. The number of connected components in G is $(V - E)$. Only option (A) is false.

117. Consider the following tree

If the post order traversal gives $ab-cd^{*+}$ then the label of the nodes 1,2,3,... will be

- a) $+, -, *, a, b, c, d$
- b) $a, -, b, +, c, *, d$
- c) $a, b, c, d, -, *, +$
- d) $-, a, b, +, *, c, d$

Explanation

Postorder traversal of the given binary tree will give the following sequence: 4 5 2 6 7 3 1. Now comparing the sequence with $a b - c d * +$ we get $1 = +$, $2 = -$, $3 = *$, $4 = a$, $5 = b$, $6 = c$ and $7 = d$. So, option (A) is correct.

118. Choose the equivalent prefix form of the following expression $(a + (b - c))^* ((d - e)/(f + g - h))$

a) **$+a - bc / - de - + fgh$**

b) $+a - bc - /de - + fgh$

c) $+a - bc / - ed + - fgh$

d) $+ab - c / - ed + - fgh$

Explanation

We can write the prefix form of the expression as:

$(a + (b - c))^* ((d - e) / (f + g - h)) =$

$(a + (- b c)) * ((- d e) / (+ f g - h)) =$

$(+ a - b c) * ((- d e) / (- + f g h)) =$

$(+ a - b c) * (/ - d e - + f g h) =$

$* + a - b c / - d e - + f g h$

119. The inorder and preorder Traversal of binary Tree are dbeafcg and abdecfg respectively. The post-order Traversal is _____.

a) dbefacg

b) debfagc

c) dbefcga

d) **debfgca**

Explanation

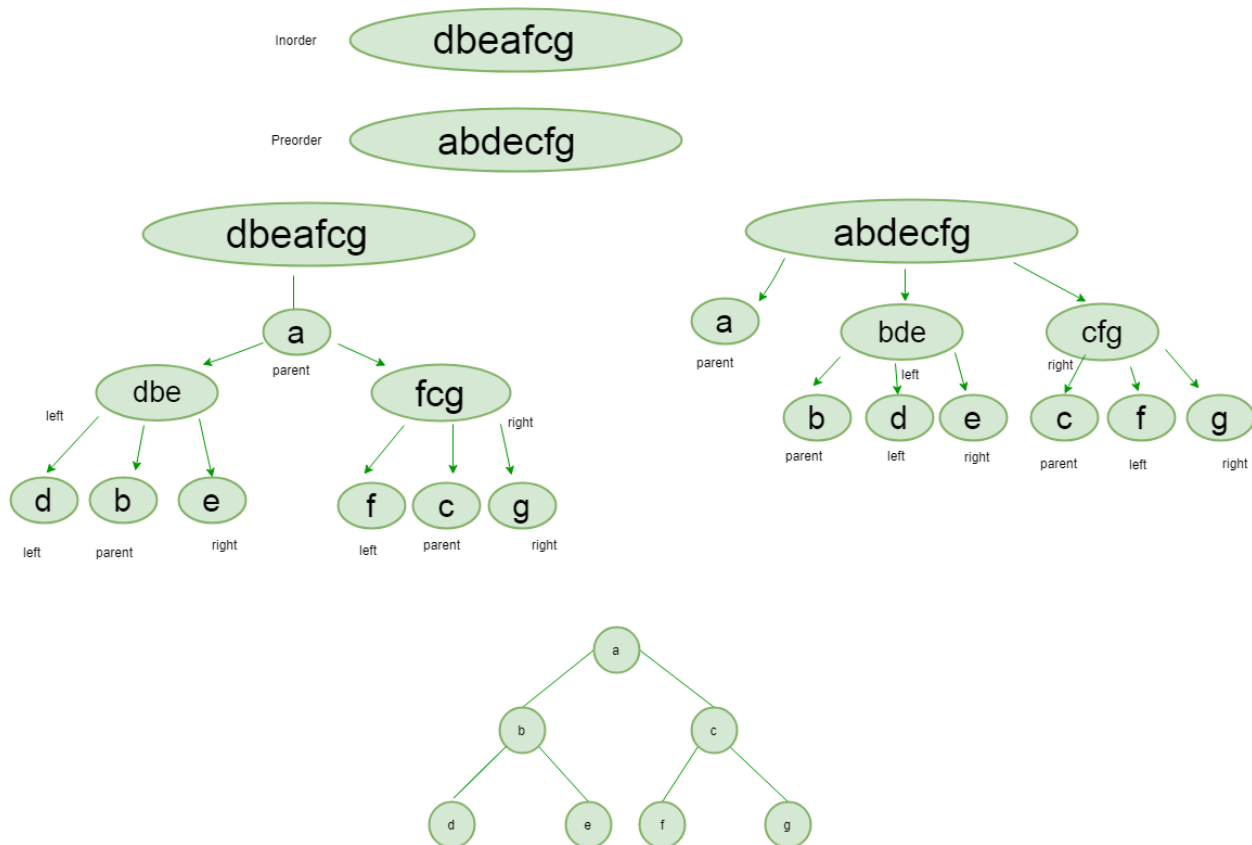
Inorder and preorder Traversal of binary Tree are dbeafcg and abdecfg respectively.

From preorder(Parent left right) and inorder (left parent right) we can easily find post order.

From preorder(a(bdecfg)), it is clear that a is parent node(root node).

Now we will look for left subtree in inorder traversal i.e. dbe and fcg.

To find root node and left subtree and right subtree of these subtree we will do the same process as above:
Now see this scenario in graph:



Now from above tree we can easily find out the post order(left right parent): i.e. debfgca. So, option (D) is correct.

120. Level order Traversal of a rooted Tree can be done by starting from root and performing:

a) Breadth First Search

b) Depth First Search

c) Root Search

d) Deep Search

Explanation

Level order Traversal of a rooted Tree can be done by starting from root and performing Breadth First Search. For more information on Level order Traversal Refer: [Level Order Tree Traversal](#) Option (A) is correct.

121. The in-order traversal of a tree resulted in FBGADCE. Then the pre-order traversal of that tree would result in

a) FGBDECA

b) ABFGCDE

c) BFGCDEA

d) AFGBDEC

122. Which of the following tree traversal uses a queue data structure?

a) Preorder

b) Inorder

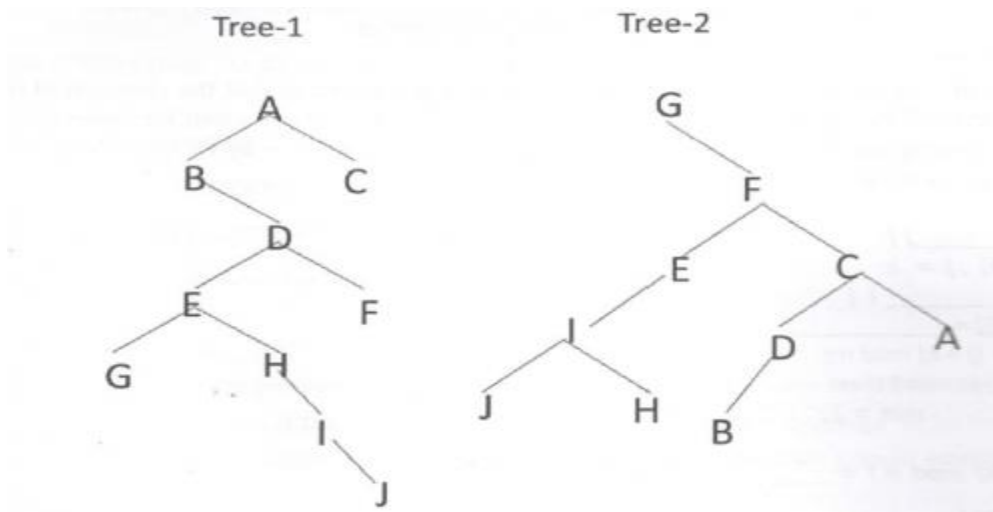
c) Postorder

d) Level order

Explanation

[Level order traversal](#) uses a queue data structure to visit the nodes level by level.

123. If Tree-1 and Tree-2 are the trees indicated below:



Which traversals of Tree-1 and Tree-2, respectively, will produce the same sequence?

- a) Preorder, postorder
- b) Postorder, inorder**
- c) Postorder, preorder
- d) Inorder, preorder

124. Which of the following cannot generate the full binary tree?

- a) Inorder and Preorder
- b) Inorder and Postorder
- c) Preorder and Postorder
- d) None of the above**

Explanation

To generate a binary tree, two traversals are necessary and one of them must be inorder.

But, a full binary tree can be generated from preorder and postorder traversals.?

125. The array representation of a complete binary tree contains the data in sorted order. Which traversal of the tree will produce the data in sorted form?

- a) Preorder
- b) Inorder
- c) Postorder
- d) Level order**

Explanation

The [level order traversal](#) of a binary tree prints the data in the same order as it is stored in the array representation of a complete binary tree.

126. What is common in three different types of traversals (Inorder, Preorder and Postorder)?

- a) Root is visited before right subtree
- b) Left subtree is always visited before right subtree**
- c) Root is visited after left subtree
- d) All of the above
- e) None of the above

Explanation

The order of inorder traversal is LEFT ROOT RIGHT The order of preorder traversal is ROOT LEFT RIGHT The order of postorder traversal is LEFT RIGHT ROOT In all three traversals, LEFT is traversed before RIGHT

127. The inorder and preorder traversal of a binary tree are d b e a f c g and a b d e c f g, respectively. The postorder traversal of the binary tree is:

a) **d e b f g c a**

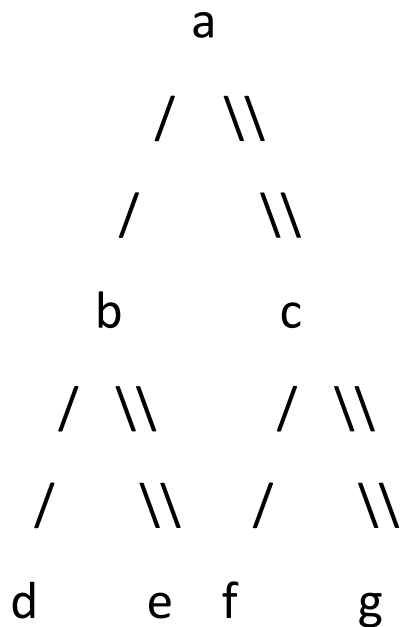
b) e d b g f c a

c) e d b f g c a

d) d e f g b c a

Explanation

Below is the given tree.



128. Which of the following pairs of traversals is not sufficient to build a binary tree from the given traversals?

a) Preorder and Inorder

b) Preorder and Postorder

c) Inorder and Postorder

d) None of the Above

129. What does the following function do for a given binary tree?

```
int fun(struct node *root)
{
    if (root == NULL)
        return 0;
    if (root->left == NULL && root->right == NULL)
        return 0;
    return 1 + fun(root->left) + fun(root->right);
}
```

a) Counts leaf nodes

b) Counts internal nodes

c) Returns height where height is defined as number of edges on the path from root to deepest node

d) Return diameter where diameter is number of edges on the longest path between any two nodes.

Explanation

The function counts internal nodes.

1) If root is NULL or a leaf node, it returns 0.

2) Otherwise returns 1 plus count of internal nodes in left subtree, plus count of internal nodes in right subtree.

[See the following complete program.](#)

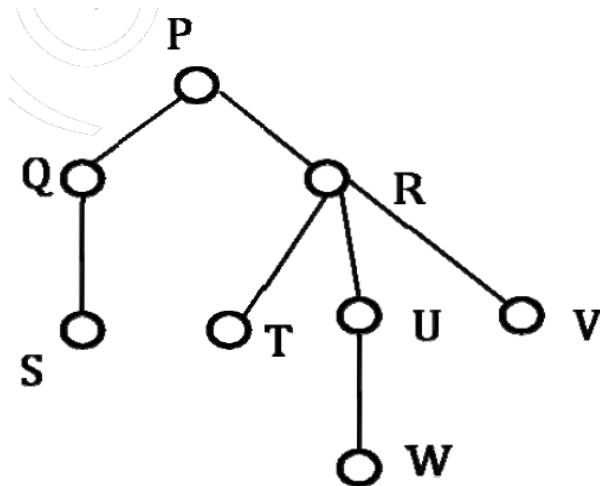
```

#include <stdio.h>
struct node
{
    int key;
    struct node *left, *right;
};
int fun(struct node *root)
{
    if (root == NULL)
        return 0;
    if (root->left == NULL && root->right == NULL)
        return 0;
    return 1 + fun(root->left) + fun(root->right);
}
/* Helper function that allocates a new node with the
   given key and NULL left and right pointers. */
struct node* newNode(int key)
{
    struct node* node = (struct node*)
                        malloc(sizeof(struct node));
    node->key = key;
    node->left = NULL;
    node->right = NULL;

    return(node);
}

```

130. Consider the following rooted tree with the vertex P labeled as root



The order in which the nodes are visited during in-order traversal is

- a) **SQPTRWUV**
- b) SQPTURWV
- c) SQPTWUVR
- d) SQPTRUWV

131. Level order traversal of a rooted tree can be done by starting from the root and performing

- a) preorder traversal
- b) inorder traversal
- c) depth first search
- d) **breadth first search**

132. Consider the label sequences obtained by the following pairs of traversals on a labeled binary tree. Which of these pairs identify a tree uniquely

- (i) preorder and postorder
- (ii) inorder and postorder
- (iii) preorder and inorder
- (iv) level order and postorder

a) Only (i)

b) (ii), (iii)

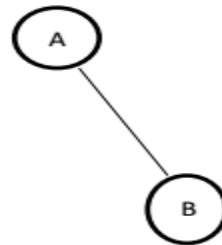
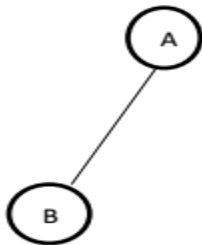
c) Only (iv)

d) Only (ii)

Explanation

Here, we consider each and every option to check whether it is true or false.

1) Preorder and postorder



For the above trees,

Preorder is AB

Postorder is BA

It shows that preorder and postorder can't identify a tree uniquely.

2) Inorder and postorder define a tree uniquely

3) Preorder and Inorder also define a tree uniquely

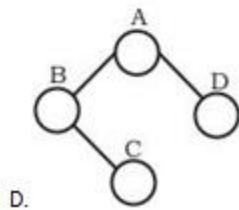
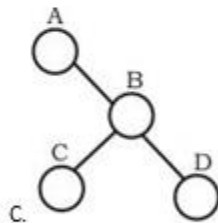
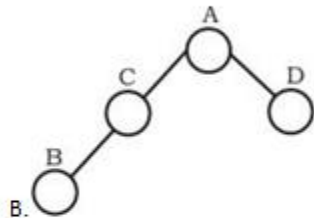
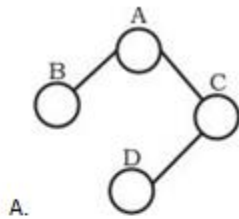
4) Levelorder and postorder can't define a tree uniquely.

For the above example,

Level order is AB

Postorder is BA

133. Which one of the following binary trees has its inorder and preorder traversals as BCAD and ABCD, respectively?



- a) A
- b) B
- c) C
- d) D**

Explanation

Inorder Traversal:

Left -Root -Right

PreOrder Traversal:

Root-Left-Right

InOrder PreOrder

- A. BADC ABCD
- B. BCAD ACBD
- C. ACBD ABCD
- D. BCAD ABCD**

Therefore, D is Correct

134. Let LASTPOST, LASTIN and LASTPRE denote the last vertex visited in a postorder, inorder and preorder traversal, respectively, of a complete binary tree. Which of the following is always true?

- a) LASTIN = LASTPOST
- b) LASTIN = LASTPRE
- c) LASTPRE = LASTPOST
- d) None of the above**

135. Which of these is an example of instance simplification approach?

- a) Presorting**
- b) Heap sort
- c) Heapify
- d) Least common mutiple

136. Which of these is not a variation of Transform and Conquer? (Tick all that applies)

- a) Instance simplification
- b) Problem transformation**
- c) Problem reduction
- d) Representation change

137. What are the three major variations of Transform and Conquer problems?

- a) Instance Expansion, Representation Simplification, Problem Expansion
- b) Instance Simplification, Representation Change, Problem Reduction**
- c) Brute Force, Backtracking, Branch and Bound
- d) Divide and Conquer, Dynamic Programming, Greedy Algorithms

138. In the context of Transform and Conquer, what does 'Instance Simplification' involve?

- a) Dividing the problem into smaller subproblems
- b) Transforming the problem into an instance of a different problem
- c) Transforming the problem into a simpler or more convenient instance of the same problem**
- d) Changing the representation of the problem instance

139. What is the time complexity of computing the mode with brute force?

- a) $\Theta(\log n)$
- b) $\Theta(n)$
- c) $\Theta(n \log n)$
- d) $\Theta(n^2)$**

140. What is the time complexity of presorting an array using an efficient sorting algorithm, in the context of Transform and Conquer?

- a) $\Theta(\log n)$
- b) $\Theta(n \log n)$**
- c) $\Theta(n)$
- d) $\Theta(n^2)$

141. In the Presort Mode algorithm, what is the primary advantage of presorting the array before computing the mode?

- a) It increases the average value of the elements in the array
- b) It makes all equal values adjacent to each other**
- c) It eliminates duplicate values from the array
- d) It reduces the space complexity of the algorithm

Explanation

Presorting ensures equal values are adjacent, simplifying the search for the longest run of equal values.

142. Which of the following is NOT a principal variety of the transform and conquer strategy?

- a) Problem reduction
- b) Instance simplification
- c) Divide and conquer**
- d) Representation change

143. What does instance simplification involve in the transform and conquer strategy?

- a) Transforming to an instance with special properties making it easier to solve**
- b) Changing the representation of the instance
- c) Applying a known algorithm directly
- d) Transforming a problem into a different problem

144. What is the role of representation change in the transform and conquer strategy?

- a) Reducing the problem size
- b) Simplifying the instance of the problem
- c) Changing one representation of an instance to another representation of the same instance**
- d) Transforming a problem into a different problem

145. What is a Binary Search Tree (BST)?

- a) A type of linked list
- b) A type of tree data structure**
- c) A type of graph
- d) A type of array

Explanation

A Binary Search Tree is a type of tree data structure where each node has at most two children.

146. Which property does a Binary Search Tree satisfy?

- a) Left child $>$ Parent $<$ Right child
- b) Left child $<$ Parent $<$ Right child**
- c) Left child = Parent = Right child
- d) None of the above

Explanation

In a Binary Search Tree, the left child is less than the parent and the right child is greater than the parent.

147. What is the average time complexity for search operations in a balanced BST?

- a) $O(1)$
- b) $O(\log n)$**
- c) $O(n)$
- d) $O(n \log n)$

Explanation

The average time complexity for search operations in a balanced Binary Search Tree is $O(\log n)$.

148. What is the time complexity of searching an element in a balanced BST with n nodes?

- a) $O(n)$
- b) $O(\log n)$**
- c) $O(n \log n)$
- d) $O(1)$

Explanation

In a balanced BST, height $\approx \log_2(n)$, so search takes $O(\log n)$ time.

149. Which traversal method is used to retrieve the values of a BST in sorted order?

- a) Pre-order
- b) Post-order
- c) In-order**
- d) Level-order

150. What happens if you insert a duplicate value in a Binary Search Tree?

- a. It replaces the existing value
- b. It is ignored
- c. It causes an error
- d. Both A and B are possible**

Explanation

In many implementations, duplicates can be ignored or handled in specific ways, such as placing them in a secondary structure.

151. What is the successor of the maximum element in a BST?

- a) Root
- b) Null**
- c) Minimum element
- d) Leftmost node

Explanation

The maximum element is the rightmost node, which has no successor in BST order. So, the successor is null.

152. In a BST, which node has the highest key?

- a) Root
- b) Rightmost leaf**
- c) Leftmost leaf
- d) Internal node

Explanation

The value of node increases as we go to its right child. So, answer = Rightmost leaf.

153. In a BST, the smallest element is found at:

- a) Root node
- b) Leftmost node**
- c) Rightmost node
- d) Any leaf node

Explanation

The value decreases as we go in the left child. Thus, smallest key is always found by going leftmost from the root. Hence, leftmost node.

154. Deleting a node with two children in a BST involves:

- a) Replacing it with the inorder predecessor or successor**
- b) Replacing it with any child
- c) Deleting both subtrees
- d) Rebuilding the tree

Explanation

When deleting a node with two children, replace it with its inorder predecessor or successor to maintain BST order.

155. Which traversal of a BST yields keys in ascending order?

- a) Preorder
- b) Inorder**
- c) Postorder
- d) Level Order

Explanation

Inorder traversal means: Visit Left $\rightarrow$ Root $\rightarrow$ Right. So, at each node, we first visit everything smaller (left side), then the node itself, then everything larger (right side).

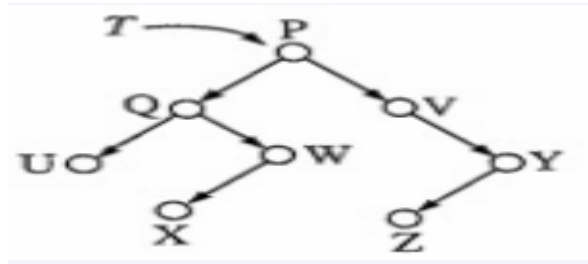
156. In a complete binary tree with n nodes, the number of leaf nodes is approximately:

- a) $n/4$
- b) $n/2$**
- c) n
- d) $\log n$

Explanation

In a complete binary tree (heap), roughly half of the nodes are leaves.

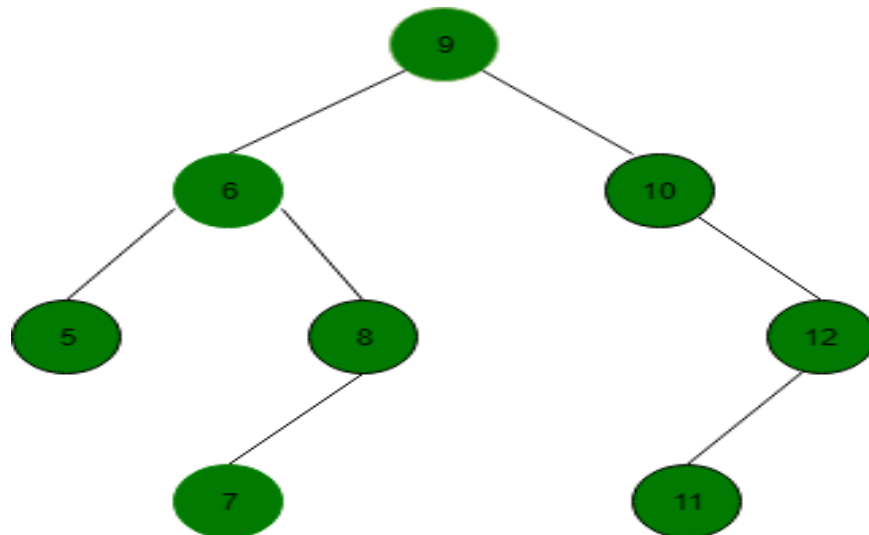
157. Consider the following binary search tree T given below: Which node contains the fourth smallest element in T?



- a) Q
- b) V
- c) W**
- d) X

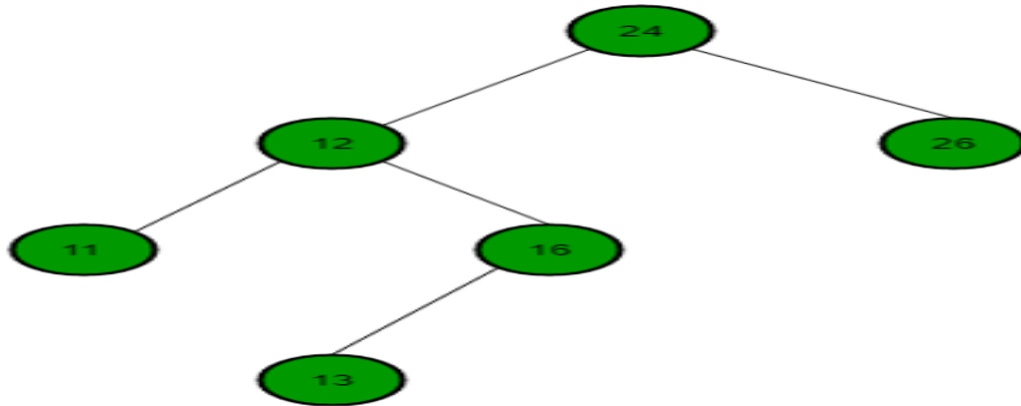
Explanation

In a BST, $\text{value}(\text{left node}) < \text{value}(\text{root}) < \text{value}(\text{right node})$. We can consider an example to find out the 4th smallest element:



From the above BST, we can clearly see that 8 is the fourth smallest element. So, option (C) is correct.

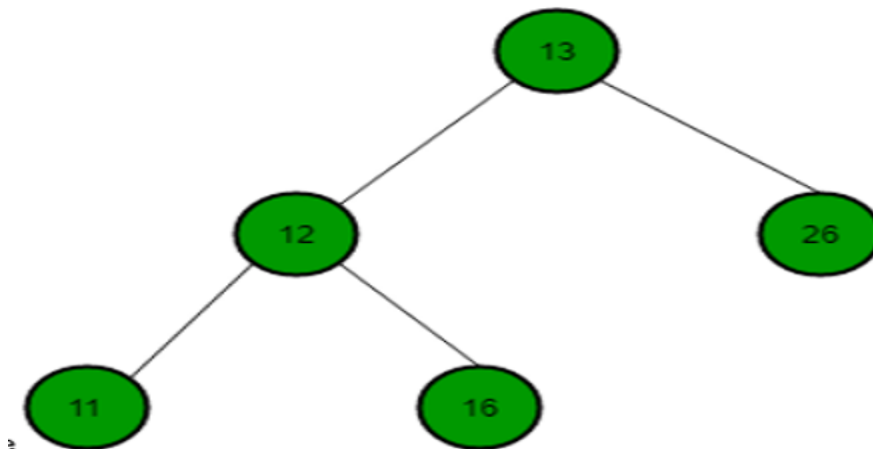
158. Consider the following binary search tree:
If we remove the root node, which of the node from the left subtree will be the new root?



- a) 11
- b) 12
- c) 13
- d) 16**

Explanation

When we delete root node, tree will be



but this tree is not balance, so we will find inorder predecessor or inorder successor and replace it with root node. In this problem we will use inorder predecessor. Root node will be 16. So, option (D) is correct.

159. The following numbers are inserted into an empty binary search tree in the given order: 10, 1, 3, 5, 15, 12, 16. What is the height of the binary search tree (the height is the maximum distance of a leaf node from the root)?

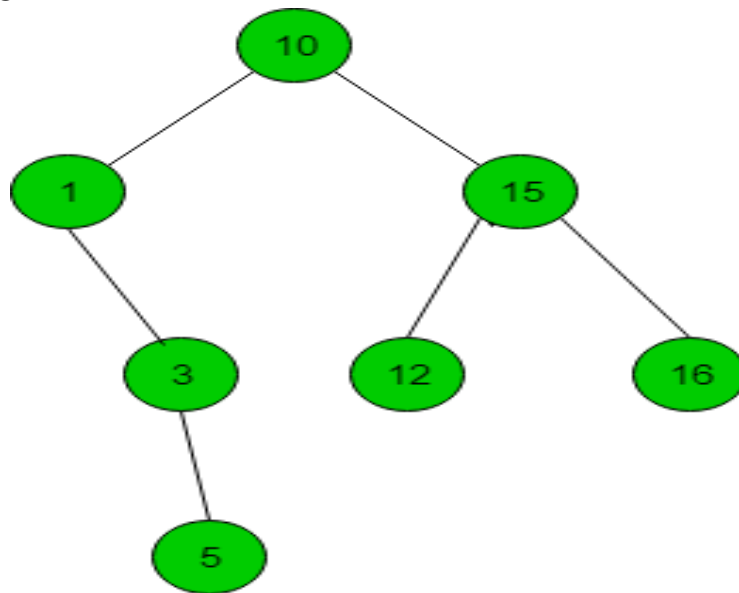
a) 2

b) 3

c) 4

d) 6

Explanation



So, height of the tree is 3, option (B) is correct.

160. The postorder traversal of a binary tree is 8, 9, 6, 7, 4, 5, 2, 3, 1. The inorder traversal of the same tree is 8, 6, 9, 4, 7, 2, 5, 1, 3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is _____ .

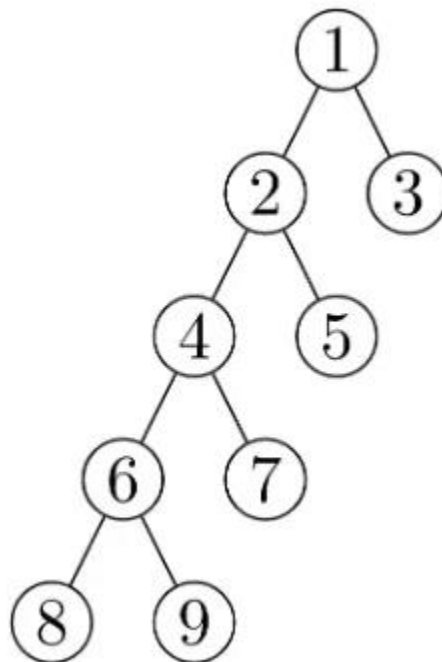
- a)2
- b)3
- c)4**
- d)5

Explanation

Given, post-order - 8, 9, 6, 7, 4, 5, 2, 3, 1

and in-order - 8, 6, 9, 4, 7, 2, 5, 1, 3

Construct a binary tree from postorder and inorder traversal:



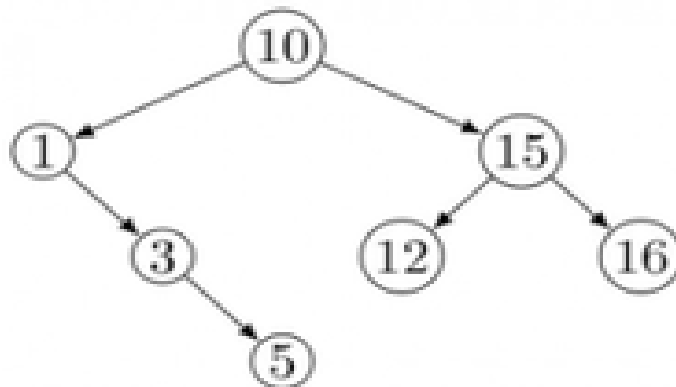
The height of the binary tree above is 4.

161. The following numbers are inserted into an empty binary search tree in the given order: 10, 1, 3, 5, 15, 12, 16. What is the height of the binary search tree (the height is the maximum distance of a leaf node from the root)?

- a) 2
- b) **3**
- c) 4
- d) 6

Explanation

Constructed binary search tree will be:



162. Suppose that we have numbers between 1 and 1,000 in a binary search tree and want to search for the number 364. Which of the following sequences could not be the sequence of nodes examined?

- a) 925, 221, 912, 245, 899, 259, 363, 364
- b) 3, 400, 388, 220, 267, 383, 382, 279, 364
- c) 926, 203, 912, 241, 913, 246, 364**
- d) 3, 253, 402, 399, 331, 345, 398, 364

Explanation

We have to find 364 in BST:

- In first option 925 is root node, our key is less than 925 so we go for left BST. Next node is 221 → 912 → 245 → 899 → 259 → 363 → 364 respectively.
- In second option 3 is root node, we go for right BST i.e. 400 → 388 → 220 → 267 → 383 → 382 → 279 → 364 respectively.
- In third option 926 is root node, we go for left BST i.e. 203 → 912 → 241 next key is 913 we can't go for 913 after 241 because we are already in left BST of 912 our key will be surely in left BST of 912. This option is incorrect.
- In fourth option 3 is root node, we go for right BST i.e. 253 → 402 → 399 → 331 → 345 → 398 → 364.

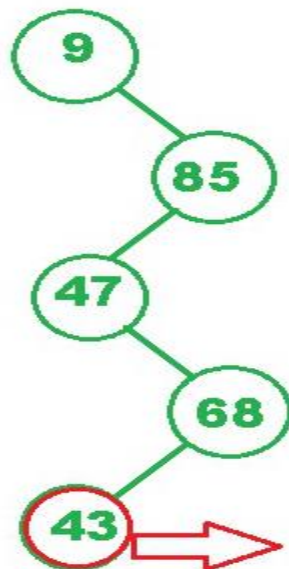
So, option (C) is correct.

163. Suppose that we have numbers between 1 and 100 in a binary search tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?

- a) {10, 75, 64, 43, 60, 57, 55}
- b) {90, 12, 68, 34, 62, 45, 55}
- c) {9, 85, 47, 68, 43, 57, 55}**
- d) {79, 14, 72, 56, 16, 53, 55}

Explanation

In BST, on right child of parent should be greater than parent and left child should be smaller than the parent, but in C after 47, 68 goes on the right side because it is greater than parent, now everything below this point should be greater than 47 but 43 appears that does not satisfy the BST property.



**Here 43 is not greater than 47
but its present on the right
side.
Not Possible !**

164. The preorder traversal of a binary search tree is 15, 10, 12, 11, 20, 18, 16, 19. Which one of the following is the postorder traversal of the tree?

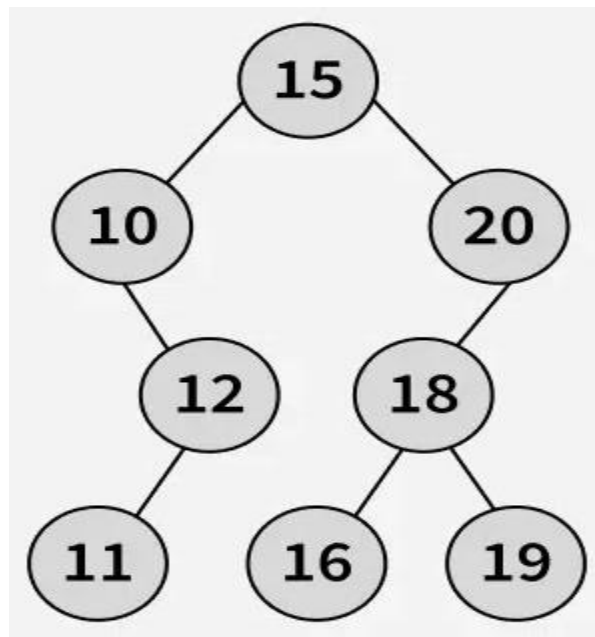
- a) 10, 11, 12, 15, 16, 18, 19, 20
- b) 11, 12, 10, 16, 19, 18, 20, 15**
- c) 20, 19, 18, 16, 15, 12, 11, 10
- d) 19, 16, 18, 20, 11, 12, 10, 15

Explanation

The given preorder is - 15, 10, 12, 11, 20, 18, 16, 19.

The inorder traversal of a binary search tree (BST) results in a sorted list.

So, the inorder traversal will be - 10, 11, 12, 15, 16, 18, 19, 20



BST

By taking help of both the traversal, we can design the required BST.

Option (B) is correct.

165. Suppose the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the usual ordering on natural numbers. What is the in-order traversal sequence of the resultant tree?

- a) 7 5 1 0 3 2 4 6 8 9
- b) 0 2 4 3 1 6 5 9 8 7
- c) 0 1 2 3 4 5 6 7 8 9**
- d) 9 8 6 4 2 3 0 1 5 7

Explanation

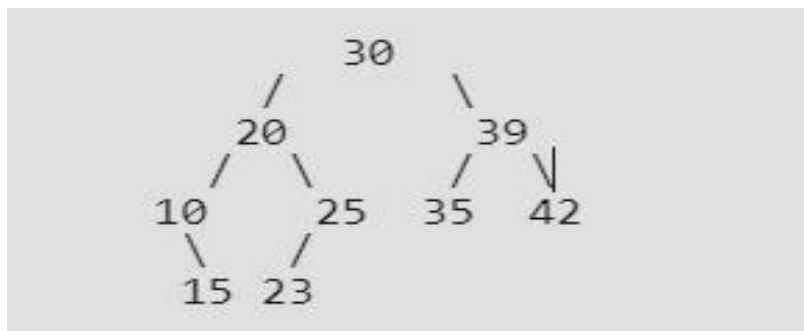
In-order traversal of a BST gives elements in increasing order. So, answer c is correct.

166. The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the postorder traversal sequence of the same tree?

- a) 10, 20, 15, 23, 25, 35, 42, 39, 30
- b) 15, 10, 25, 23, 20, 42, 35, 39, 30
- c) 15, 20, 10, 23, 25, 42, 35, 39, 30
- d) 15, 10, 23, 25, 20, 35, 42, 39, 30**

Explanation

The following is the constructed tree



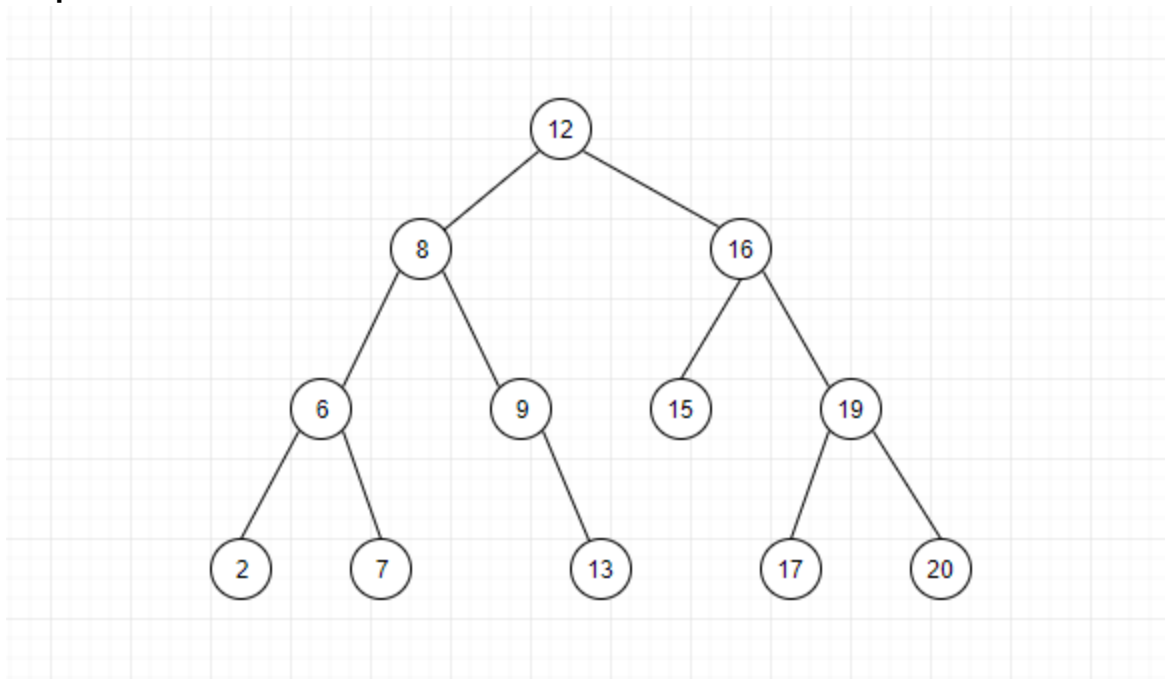
167. The post-order traversal of a binary search tree is given by 2, 7, 6, 10, 9, 8, 15, 17, 20, 19, 16, 12. Then the pre-order traversal of this tree is:

- a) 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 19, 20
- b) 7, 6, 2, 10, 9, 8, 15, 16, 17, 20, 19, 12
- c) 7, 2, 6, 8, 9, 10, 20, 17, 19, 15, 16, 12
- d) 12, 8, 6, 2, 7, 9, 10, 16, 15, 19, 17, 20**

Explanation

Since given tree is binary tree, so inorder traversal will be always sorted order, i.e., 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 19, 20.

Now we can draw that binary search tree using given postorder and inorder traversal. Final tree will be:



Therefore, preorder traversal will be: 12, 8, 6, 2, 7, 9, 10, 16, 15, 19, 17, 20. Option (D) is correct.

168. Suppose the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the reversal ordering on natural numbers i.e. 9 is assumed to be smallest and 0 is assumed to be largest. The in-order traversal of the resultant binary search tree is

- a) 9, 8, 6, 4, 2, 3, 0, 1, 5, 7
- b) 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- c) 0, 2, 4, 3, 1, 6, 5, 9, 8, 7
- d) 9, 8, 7, 6, 5, 4, 3, 2, 1, 0**

Explanation

Inorder traversal of a binary search tree always produces the keys in increasing order. In this question Reverse ordering of natural numbers are used i.e. 9 is assumed to be the smallest and 0 to be the largest. So the sequence in increasing order will be 9, 8, 7, 6, 5, 4, 3, 2, 1, 0. So, option (D) is correct.

169. Which of the following traversals is sufficient to construct BST from given traversals 1) Inorder 2) Preorder 3) Postorder

- a) Any one of the given three traversals is sufficient
- b) Either 2 or 3 is sufficient**
- c) 2 and 3
- d) 1 and 3

Explanation

When we know either preorder or postorder traversal, we can construct the BST. Note that we can always sort the given traversal and get the inorder traversal. Inorder traversal of BST is always sorted.

170. What are the worst-case complexities of insertion and deletion of a key in a binary search tree?

- a) $\Theta(\log n)$ for both insertion and deletion
- b) $\Theta(n)$ for both insertion and deletion**
- c) $\Theta(n)$ for insertion and $\Theta(\log n)$ for deletion
- d) $\Theta(\log n)$ for insertion and $\Theta(n)$ for deletion

Explanation

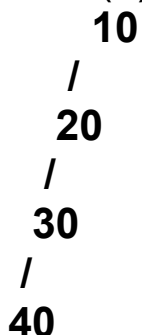
The time taken by search, insert and delete on a BST is always proportional to height of [BST](#). Height may become $O(n)$ in worst case.

171. What is the worst-case time complexity for search, insert and delete operations in a general Binary Search Tree for a skewed tree?

- a) $O(n)$ for all**
- b) $O(\log n)$ for all
- c) $O(\log n)$ for search and insert, and $O(n)$ for delete
- d) $O(\log n)$ for search, and $O(n)$ for insert and delete

Explanation

In skewed Binary Search Tree (BST), all three operations can take $O(n)$. See the following example BST and operations.



Search 40.

Delete 40

Insert 50.

172. What is the height of an essentially complete binary tree with n nodes?

- a) **log base 2 of n**
- b) square root of n
- c) $n \log$ base 2 of n
- d) n squared

173. While inserting the elements 71, 65, 84, 69, 67, 83 in an empty binary search tree (BST) in the sequence shown, the element in the lowest level is

- a) 65
- b) **67**
- c) 69
- d) 83

Explanation

Here is The Insertion Algorithm for a Binary Search Tree:

```
Insert(Root,key)
{
    if(Root is NULL)
        Create a Node with value as key and return
    Else if(Root.key >= key)
        Insert(Root.left,key)
    Else
        Insert(Root.right,key)
}
```

174. A Binary Search Tree (BST) stores values in the range 37 to 573. Consider the following sequence of keys.

- I. 81, 537, 102, 439, 285, 376, 305
- II. 52, 97, 121, 195, 242, 381, 472
- III. 142, 248, 520, 386, 345, 270, 307
- IV. 550, 149, 507, 395, 463, 402, 270

Which of the following statements is TRUE?

- a) I, II and IV are inorder sequences of three different BSTs
- b) I is a preorder sequence of some BST with 439 as the root
- c) II is an inorder sequence of some BST where 121 is the root and 52 is a leaf**
- d) IV is a postorder sequence of some BST with 149 as the root

Explanation

A: I and IV are not in ascending order

B: If 439 is root, it should be 1st element in preorder

D: If 149 is root, it should be last element in postorder.

175. What does AVL stand for in AVL Tree?

- a. Adelson-Velsky and Landis**
- b. Advanced Value List
- c. Algebraic Variable Logic
- d. Automated Vector List

Explanation

AVL stands for Adelson-Velsky and Landis, named after its inventors.

176. What is an AVL tree?

- a) **a tree which is balanced and is a height balanced tree**
- b) a tree which is unbalanced and is a height balanced tree
- c) a tree with three children
- d) a tree with at most 3 children

Explanation

It is a self-balancing tree with height difference at most 1.

177. What is the primary purpose of an AVL tree?

- a. To store data sequentially
- b. To maintain balance in binary search trees**
- c. To sort data efficiently
- d. To implement graph algorithms

Explanation

The primary purpose of an AVL tree is to maintain balance in binary search trees for efficient data retrieval.

178. How does an AVL tree maintain balance?

- a. By using a simple linked list
- b. Through rotations**
- c. By deleting nodes
- d. By increasing node values

Explanation

An AVL tree maintains balance through rotations which adjust the tree structure to keep it balanced.

179. What is the maximum height difference (balance factor) allowed between the left and right subtrees in an AVL tree?

- a. 1
- b. 2
- c. 3
- d. 4

Explanation

The maximum height difference allowed in an AVL tree is 1 for it to be balanced.

180. Which of the following is TRUE?

- a) **The cost of searching an AVL tree is $\theta(\log n)$ but that of a binary search tree is $O(n)$**
- b) The cost of searching an AVL tree is $\theta(\log n)$ but that of a complete binary tree is $\theta(n \log n)$
- c) The cost of searching a binary search tree is $O(\log n)$ but that of an AVL tree is $\theta(n)$
- d) The cost of searching an AVL tree is $\theta(n \log n)$ but that of a binary search tree is $O(n)$

Explanation

AVL tree is a balanced tree.

AVL tree's time complexity of searching = $\theta(\log(n))$.

But a binary search tree may be a skewed tree, so in the worst-case BST searching time = $\theta(n)$

181. Which operation is performed when an AVL tree becomes unbalanced due to insertion?

- a. Left Rotation
- b. Right Rotation
- c. Both Left and Right Rotations**
- d. No operation needed

Explanation

When an AVL tree becomes unbalanced due to insertion, either a Left Rotation, Right Rotation, or both may be performed to restore balance.

182. To restore the AVL property after inserting an element, we start at the insertion point and move towards root of that tree. is this statement true?

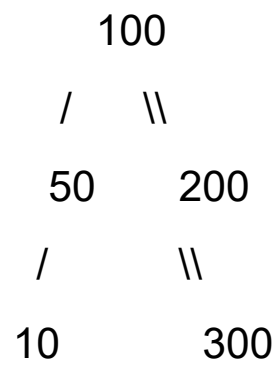
- a) true**
- b) false

Explanation

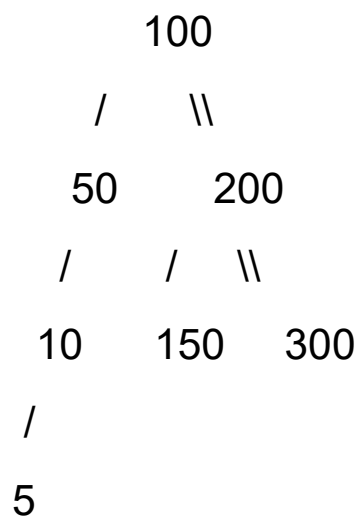
It is interesting to note that after insertion, only the path from that point to node or only that subtrees are imbalanced in terms of height.

183. Which of the following is AVL Tree?

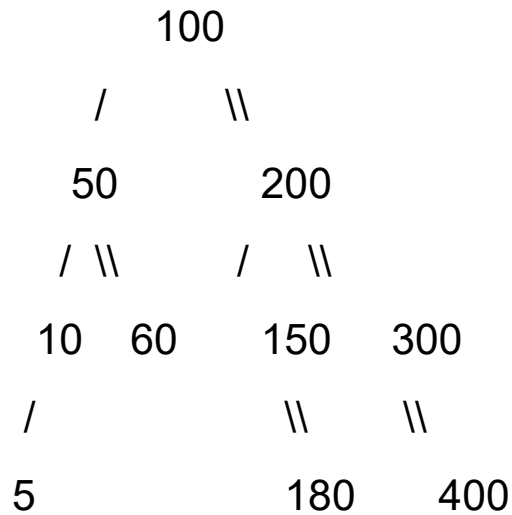
a)



b)



c)



a) Only A

b) A and C

c) A, B and C

d) Only B

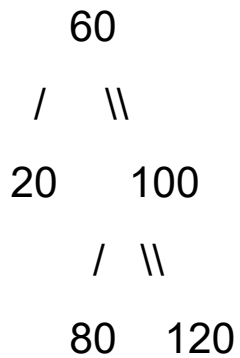
Explanation

A Binary Search Tree is AVL if balance factor of every node is either -1 or 0 or 1.

Balance factor of a node X is [(height of X->left) - (height of X->right)].

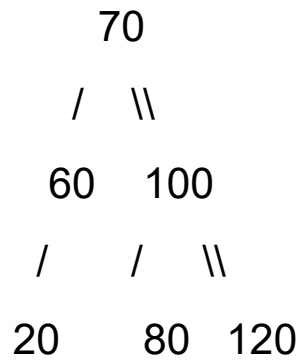
In Tree B, the node with value 50 has balance factor 2. That is why B is not an AVL tree.

184. Consider the following AVL tree.

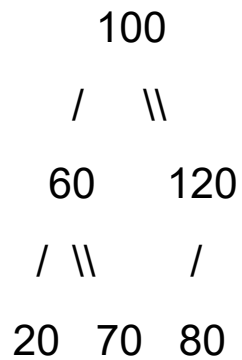


Which of the following is updated AVL tree after insertion of 70?

A



B



C

80
/ \\
60 100
/ \ \\
20 70 120

D

80
/ \\
60 100
/ / \\
20 70 120

a)A

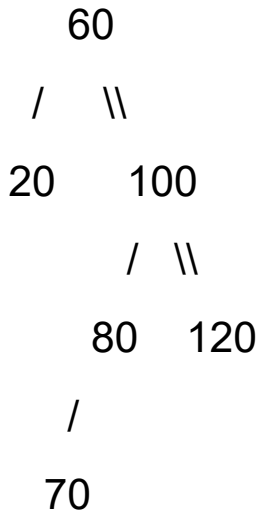
b)B

c)C

d)D

Explanation

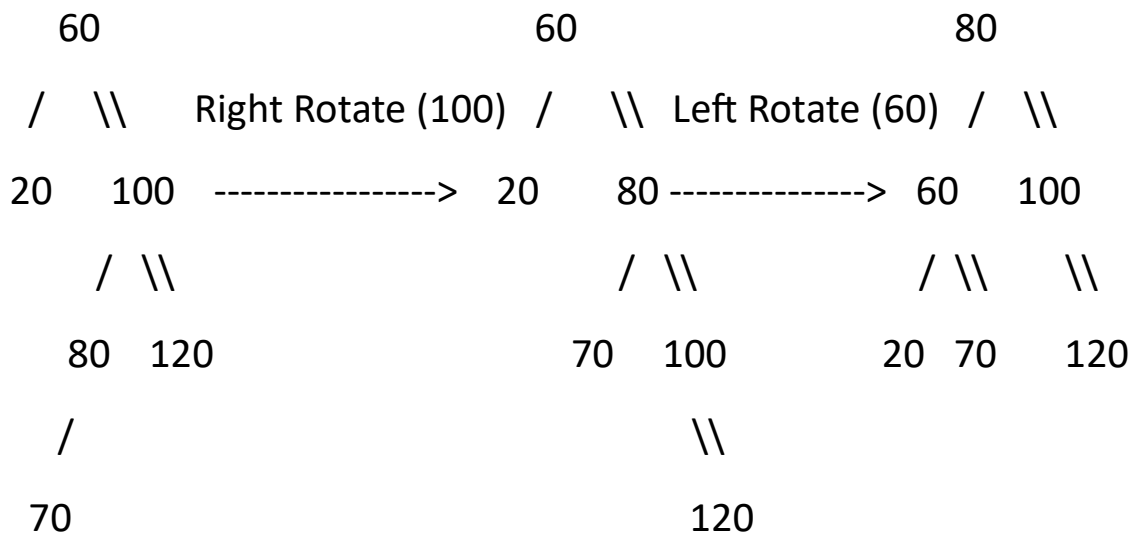
After insertion of 70, tree becomes following:



We start from 50 and travel up. We keep travelling up till we find an unbalanced node.

In above case, we reach the node 60 and see 60 got unbalanced after insertion and this is Right Left Case.

So, we need to apply two rotations

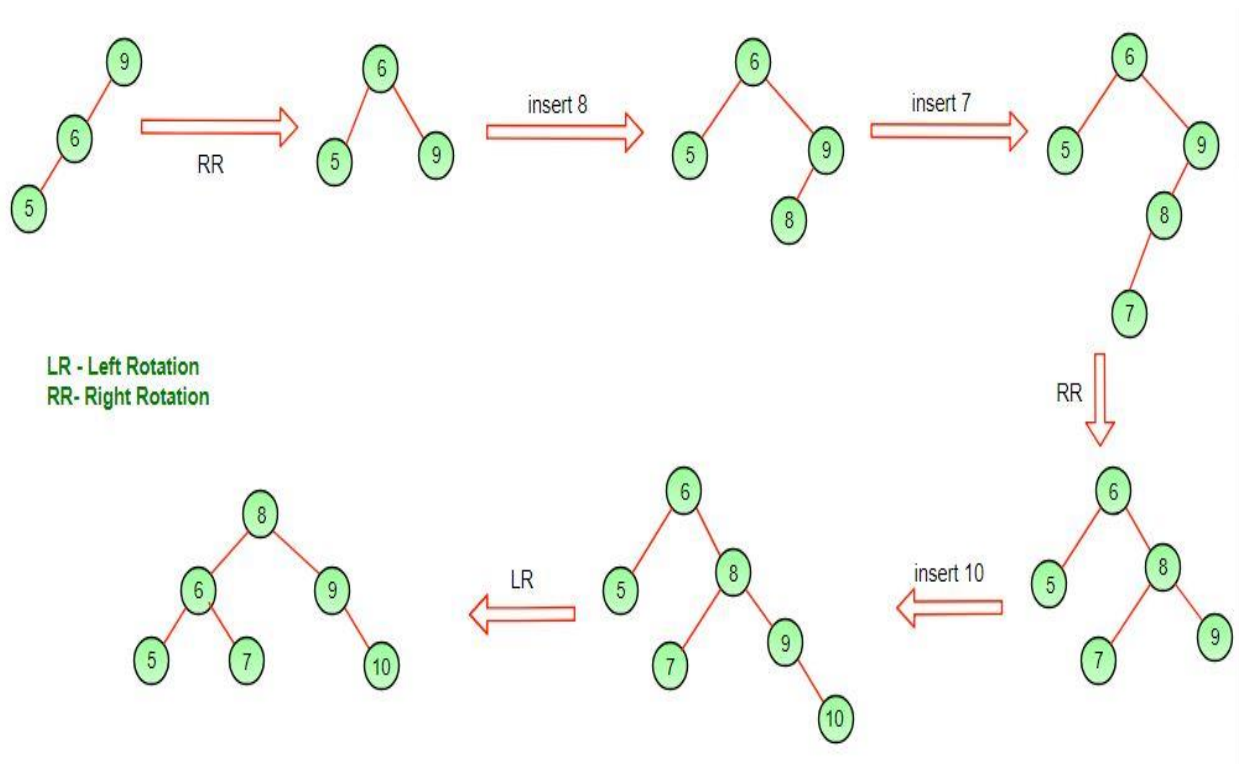


185. The number of rotations required to insert a sequence of elements 9,6,5,8,7,10 into an empty AVL tree is?

- a) 0
- b) 1
- c) 2
- d) 3

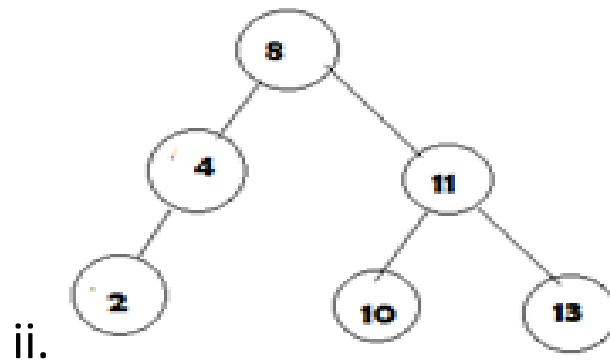
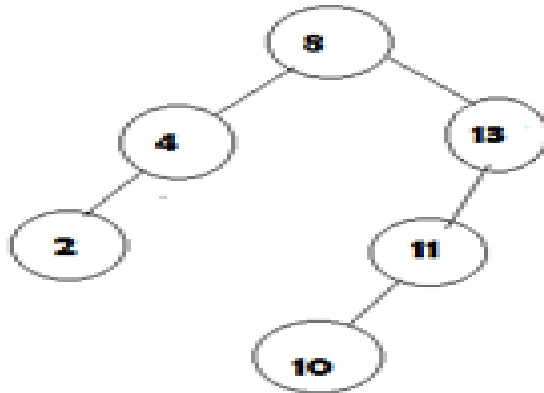
Explanation

The **insertion** and rotation of the various elements are shown in the following figure:



So, the total number of rotations are 3.

186. Which of the below diagram is following AVL tree property



- a) only i
- b) both i and ii
- c) only ii**
- d) i is not a binary search tree

Explanation

The property of AVL tree is it is height balanced tree with difference of at most 1 between left and right subtrees. All AVL trees are binary search tree.

187. Consider the pseudo code:

```
int avl(binarysearchtree root):  
    if(not root)  
        return 0  
    left_tree_height = avl(left_of_root)  
    if(left_tree_height == -1)  
        return left_tree_height  
    right_tree_height = avl(right_of_root)  
    if(right_tree_height == -1)  
        return right_tree_height
```

Does the above code can check if a binary search tree is an AVL tree?

a) yes

b) no

Explanation:

The code only calculates subtree heights and propagates -1 if a subtree fails but never checks the AVL balance condition $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$. Hence, it cannot verify if the tree is AVL.

188. What is a heap data structure primarily used for?

a. Sorting elements

b. Implementing priority queues

c. Graph representation

d. Data compression

Explanation

Heaps are typically used to implement priority queues, where the highest (or lowest) priority element can be efficiently retrieved.

189. What is the 'parental dominance' property in the context of a heap data structure?

- a) The key in each node is equal to the keys in its children
- b) The key in each node is greater than or equal to the keys in its children**
- c) The key in each node is less than or equal to the keys in its children
- d) The key in each node is unrelated to the keys in its children

190. What is the 'shape property' of a heap?

- a) The binary tree must be perfectly balanced with all levels completely full
- b) The binary tree is essentially complete, with all levels full except possibly the last, where only rightmost leaves may be missing**
- c) The binary tree can have any shape as long as the parental dominance property is maintained
- d) The binary tree must be a full binary tree, where every node has either 0 or 2 children

191. Which of the following is a characteristic of a max-heap?

- a. Parent node is greater than children**
- b. Parent node is less than children
- c. All nodes are equal
- d. Nodes are sorted in ascending order

Explanation

In a max-heap, the parent node is always greater than or equal to its child nodes.

192. Which of the following arrays represents a valid max heap?

- a) [90, 15, 10, 7, 12, 2, 7, 3]
- b) [10, 15, 30, 40, 50]
- c) [100, 19, 36, 17, 3, 25, 1, 2, 7]**
- d) [5, 7, 6, 8, 10]

Explanation

Every parent is greater than its children in (c), satisfying the max heap property.

193. What is the height of a heap with n elements?

- a) $O(n)$
- b) $O(\log n)$**
- c) $O(1)$
- d) $O(n \cdot \log n)$

Explanation

A heap is a complete binary tree, and the height of a complete binary tree is $\log_2(n)$.

194. Which of the following statements about a binary heap is TRUE?

- a) It is always a complete binary tree**
- b) It is always a balanced binary search tree
- c) It is always full
- d) It is always sorted

Explanation

A heap is complete (all levels filled except possibly the last), not necessarily sorted or BST-ordered.

195. What is the main property of a binary heap?

a) It is a complete binary tree

b) It is always balanced

c) All nodes have two children

d) It can be any shape

Explanation

A binary heap is a complete binary tree which satisfies the heap property.

196. Which of the following operations can be performed in a heap?

a. Insert

b. Delete

c. Peek

d. All of the above

197. Which of the following operations can be performed in $O(\log n)$ time on a binary heap?

a) Insert

b) Delete

c) Get maximum

d) All of the above

198. In a heap implemented as an array, where are the leaf nodes located?

a) At odd indices only

b) In the first $n/2$ positions

c) In the last $n/2$ positions

d) At even indices only

199. What is the time complexity of Build Heap operation?
(Build Heap is used to build a max(or min) binary heap from a given array. Build Heap is used in Heap Sort as a first step for sorting.)

- a) $O(n \log n)$
- b) $O(n^2)$
- c) **$O(n)$**
- d) $O(\log n)$

Explanation

Following is algorithm for building a Heap of an input array A.

```
BUILD-HEAP(A)
    heapsize := size(A);
    for i := floor(heapsize/2) downto 1
        do HEAPIFY(A, i);
    end for
END
```

Although the worst-case complexity looks like $O(n \log n)$, upper bound of time complexity is $O(n)$.

200. What is the primary goal of the 'Heap Transformation Algorithm'?

- a) **To ensure parental dominance is maintained in the heap**
- b) To sort the array in ascending order
- c) To find the minimum element in the array
- d) To reverse the order of elements in the array

201. What is the time complexity of the insertion operation when transforming an array into a heap?

- a) $O(n^2)$
- b) $O(n)$
- c) $O(1)$
- d) $O(\log n)$**

202. What is the primary purpose of heap sort?

- a) To sort data efficiently**
- b) To search for data
- c) To store data
- d) To compress data

203. What data structure is used in heap sort?

- a) Array
- b) Linked List
- c) Heap**
- d) Stack

Explanation

Heap sort utilizes a heap data structure to sort elements.

204. What is the time complexity of heap sort in the worst case?

- a. $O(n \log n)$**
- b. $O(n)$
- c. $O(\log n)$
- d. $O(n^2)$

205. What is the time complexity of Heapsort in the worst and best cases?

- a) $O(n^2)$
- b) $O(n)$
- c) $\Theta(n \log n)$**
- d) $O(\log n)$

206. Which algorithm does heap sort utilize to build the heap?

- a. Insertion Sort
- b. Selection Sort
- c. Bubble Sort
- d. Heapify**

Explanation

Heap sort uses the heapify process to build the heap.

207. Which of the following sorting algorithms in its typical implementation gives best performance when applied on an array which is sorted or almost sorted (maximum 1 or two elements are misplaced).

- a) Quick Sort
- b) Heap Sort
- c) Merge Sort
- d) Insertion Sort**

Explanation

Insertion sort takes linear time when input array is sorted or almost sorted (maximum 1 or 2 elements are misplaced).

All other sorting algorithms mentioned above will take more than linear time in their typical implementation.

208. Is heap sort a stable sorting algorithm?

- a. Yes
- b. No**

Explanation

Heap sort is not a stable sorting algorithm.

209. What is the necessary condition for a Tree to be a heap?

- a) Only the tree must be complete.
- b) Every Root value should be greater or smaller than the children's value only.
- c) The tree must be complete and Every Root value should be greater or smaller than the children's value.**
- d) None

Explanation

A Heap is a special Tree-based data structure in which the tree is a complete binary tree.

Generally, Heaps can be of two types:

1. **Max-Heap:** In a Max-Heap the key present at the root node must be greatest among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.
2. **Min-Heap:** In a Min-Heap the key present at the root node must be minimum among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree.

210. You have to sort 1 GB of data with only 100 MB of available main memory. Which sorting technique will be most appropriate?

- a. Heap sort
- b. Merge sort**
- c. Quick sort
- d. Insertion sort

Explanation

The data can be sorted using external sorting which uses merging technique. This can be done as follows:

1. Divide the data into 10 groups each of size 100.
2. Sort each group and write them to disk.
3. Load 10 items from each group into main memory.
4. Output the smallest item from the main memory to disk. Load the next item from the group whose item was chosen.
5. Loop step #4 until all items are not outputted.

The step 3-5 is called as merging technique.

211. Which of the following is not true about comparison-based sorting algorithms?

- a) The minimum possible time complexity of a comparison-based sorting algorithm is $O(n(\log(n)))$ for a random input array
- b) Any comparison-based sorting algorithm can be made stable by using position as a criteria when two elements are compared
- c) Counting Sort is not a comparison-based sorting algorithm
- d) Heap Sort is not a comparison-based sorting algorithm.**

Explanation

Heap Sort is a comparison-based sorting algorithm, as it relies on comparing elements to maintain the heap property. Hence, the statement "Heap Sort is not a comparison-based sorting algorithm" is false, making it the correct answer.

212. Match the following pairs

I. $O(\log n)$	(M) Heap sort
II. $O(n)$	(N) DFS
III. $n \log n$	(O) Binary search
IV. $O(n^2)$	(P) Selecting K^{th} smallest elements

- a) I - P, II - M, III - N, IV - O
- b) I - O, II - P, III - M, IV - N
- c) I - O, II - N, III - M, IV - P**
- d) I - O, II - N, III - P, IV - M

213. Which sorting algorithm is most efficient to sort string consisting of ASCII characters?

- a) Quick sort
- b) Heap sort
- c) Merge sort
- d) Counting sort**

Explanation

Counting sort algorithm is efficient when range of data to be sorted is fixed. In the above question, the range is from 0 to 255(ASCII range). Counting sort uses an extra constant space proportional to range of data.

214. Given an array $A[1 \dots n]$ of integers, the running time of Counting Sort is polynomial in the input size n .

- a) True
- b) False**

Explanation

Counting Sort's running time depends on the size of the numbers in the input, so it is pseudo-polynomial.

215. Counting sort is a stable, in-place sorting algorithm.

a) True

b) False

Explanation

Counting sort is stable. It is not in-place, however, since we must make additional space to store the counts of the various elements.

This space requirement grows as the size of the input increases. Additionally, we have to make a separate output array to produce the answer using counting sort.

216. How many comparisons are made when sorting the array $\text{arr} = \{1, 5, 3, 8, 2\}$ using counting sort?

a) 5

b) 7

c) 9

d) 0

Explanation:

As counting sort is an example of non-comparison sort so it is able to sort an array without making any comparison.

217. Which sorting technique is most efficient when the range of input data is not significantly greater than the number of elements to be sorted?

a) Selection sort

b) Bubble sort

c) Counting sort

d) Insertion sort

Explanation:

Time complexity of counting sort is given as $O(n+k)$ where n is the number of input elements and k is the range of input. So, if range of input is not significantly larger than number of elements in the array then it proves to be very efficient.

218. What is the auxiliary space requirement of counting sort?

- a) $O(1)$
- b) $O(n)$
- c) $O(\log n)$
- d) $O(n + k)$, where k is the range of the input**

Explanation:

Counting sort uses two extra arrays to get the input array sorted.

First array is required to store the count of all the elements which fall in the range of input data elements, so its size is k .

The second array is required to store the input elements in sorted manner, so its size is n . Thus, overall auxiliary space required becomes $O(n+k)$.

219. It is not possible to implement counting sort when any of the input elements has a negative value.

- a) True
- b) False**
- c) Only if the range of values is large
- d) Only for non-integer values

Explanation:

It is possible to extend the counting sort algorithm for negative numbers as well.

In such a case we store the minimum element at the 0th index.

220. Which of these sorting techniques is stable?

- a) Quick sort
- b) Counting sort**
- c) Heap sort
- d) Selection sort

Explanation:

Counting sort is an example of stable sorting algorithm as the elements with identical values appear in the same order in the output array as they were in the input array.

221. Which of these sorting algorithms uses the largest amount of auxiliary space for sorting?

- a) Bubble sort
- b) Counting sort**
- c) Quick sort
- d) Heap sort

222. The average time complexity of counting sort is:

- a) $O(n)$
- b) $O(n + k)$, where k represents the range of the input values**
- c) $O(n^2)$
- d) $O(n \log n)$

Explanation:

Time complexity of counting sort is $O(n+k)$ as counting the occurrence of each element in the input range takes k time and then finding the correct index value of each element in the sorted array takes n time.

223. The complexity of which sorting algorithm remains consistent across its best, average, and worst cases?

- a) Quick sort
- b) Insertion sort
- c) Counting sort**
- d) Gnome sort

224. What is an advantage of counting sort compared to the quick sort?

- a) Quick sort has lesser time complexity when the range is comparable to the number of inputs elements**
- b) Counting sort has lesser space complexity
- c) Counting sort is not a comparison-based sorting technique
- d) It has no advantage

Explanation:

Counting sort is very efficient in the cases where range is comparable to number of input elements as it performs sorting in linear time.

225. Which of these algorithms takes non-linear time for sorting?

- a) Counting sort
- b) Quick sort**
- c) Bucket sort
- d) Radix sort

226. Which of the following represents the algorithm of counting sort correctly?

a)

```
function countingSort(array, k) is
  count ← new array of k zeros
  for i = 1 to length(array) do
    count[array[i]] ← count[array[i]] + 1
  for i = 2 to k do
    count[i] ← count[i] + count[i - 1]
  for i = length(array) downto 1 do
    output[count[array[i]]] ← array[i]
    count[array[i]] ← count[array[i]] - 1
  return output
```

b)

```
function countingSort(array, k) is
  count ← new array of k zeros
  for i = 1 to length(array) do
    count[array[i]] ← count[array[i]] + 1
  for i = 1 to k do
    count[i] ← count[i] + count[i - 1]
  for i = length(array) downto 1 do
    output[count[array[i]]] ← array[i]
    count[array[i]] ← count[array[i]] - 1
  return output
```

c)

```
function countingSort(array, k) is
  count ← new array of k zeros
  for i = 1 to length(array) do
    count[array[i]] ← count[array[i]] + 1
  for i = 2 to k do
    count[i] ← count[i] + count[i - 1]
  for i = length(array) downto 1 do
    output[count[array[i]]] ← array[i]
    count[array[i]] ← count[array[i]] - 1
  return output
```

d)

```
function countingSort(array, k) is
count ← new array of k zeros
for i = 1 to length(array) do
  count[array[i]] ← count[array[i]] + 1
for i = 2 to k do
  count[i] ← count[i] + count[i - 1]
for i = length(array) downto 1 do
  output[count[array[i]]] ← array[i]
  count[array[i]] ← count[array[i]] - 1
return output
```

Explanation:

The first loop counts the number of occurrences of each element. Second loop performs prefix sum on count to determine position range where items having that key should be placed in. The third loop places each element at its correct position.

227. What is the primary advantage of Counting Sort over comparison-based sorting algorithms?

- a) It has a lower space complexity.
- b) It can sort in linear time for a fixed range of input values.**
- c) It works well with floating-point numbers.
- d) It requires fewer comparisons.

Explanation:

Counting Sort achieves $O(n + k)$ time complexity, where n is the number of elements and k is the range of input values, making it linear when k is not much larger than n , unlike comparison-based sorts which are at least $O(n \log n)$.

228. In Counting Sort, what does the counting array store?

- a) The sorted elements.
- b) The frequency of each distinct element.**
- c) The positions of elements in the original array.
- d) The cumulative sum of elements.

Explanation:

The counting array keeps track of the frequency of each value in the input array, which is used to determine the positions of elements in the output array.

229. What is the time complexity of Counting Sort in the best case?

- a) $O(1)$
- b) $O(n)$
- c) $O(n \log n)$
- d) $O(n + k)$**

Explanation:

Counting Sort's time complexity is $O(n + k)$, where n is the number of elements and k is the range of input values, and this holds true regardless of the input order, as it performs a fixed number of operations.

230. Counting Sort is most efficient when:

- a) The input array is already sorted.
- b) The range of input values is small compared to the number of elements.**
- c) The array contains negative numbers.
- d) The array has a large number of duplicates.

Explanation:

When the range of input values (k) is not significantly larger than the number of elements (n), the algorithm runs in linear time, making it efficient for such cases.

231. Which of the following is NOT a step in the Counting Sort algorithm?

- a) Building the counting array.
- b) Computing the cumulative count.
- c) Performing a binary search on the array.**
- d) Placing elements in the output array.

Explanation:

Binary search is not part of Counting Sort; the algorithm uses the counting array to directly determine positions, without any searching.

232. If an array has elements from 0 to 100 and contains 50 elements, what is the space complexity of Counting Sort?

- a) $O(1)$
- b) $O(n)$
- c) $O(k)$
- d) $O(n + k)$**

Explanation:

Counting Sort requires an auxiliary array of size $k+1$ (for values 0 to 100), plus space for the output array of size n , resulting in $O(n + k)$ space complexity.

233. Can Counting Sort be used to sort strings?

- a) Yes, directly.
- b) No, only for numbers.
- c) Yes, if strings are of fixed length and mapped to integers.**
- d) No, it requires floating-point numbers.

Explanation:

Counting Sort can be adapted for strings by mapping them to integer keys, such as using radix sort, but it isn't directly applicable without conversion.

234. What happens if the input array contains duplicate elements in Counting Sort?

- a) **Duplicates are ignored.**
- b) Duplicates are placed consecutively in the output.
- c) The algorithm fails.
- d) Duplicates require extra space.

Explanation:

Counting Sort handles duplicates by maintaining their frequencies in the counting array, ensuring they are placed in the correct order in the output array.

235. In Counting Sort, the output array is constructed by:

- a) Sorting the counting array first.
- b) **Using the cumulative counts to place elements in order.**
- c) Reversing the input array.
- d) Merging with another array.

Explanation:

The cumulative sum in the counting array helps determine the exact positions for each element in the output array, ensuring the array is sorted.

236. What is the worst-case time complexity of Counting Sort?

- a) $O(n^2)$
- b) $O(n \log n)$
- c) **$O(n + k)$**
- d) $O(k \log k)$

Explanation:

Even in the worst case, Counting Sort runs in $O(n + k)$ time, as long as k is reasonable, due to its non-comparative nature.

237. Counting Sort assumes that:

- a) All elements are unique.
- b) The input is already sorted.
- c) The input elements are integers within a known range.**
- d) The array is of even length.

Explanation:

Counting Sort requires the input to be integers (or mappable to integers) within a specific range to build the counting array effectively.

238. If k (range of input) is much larger than n (number of elements), Counting Sort becomes:

- a) More efficient than quicksort.
- b) Less efficient due to high space usage.**
- c) Ideal for large datasets.
- d) Faster than merge sort.

Explanation:

When $k \gg n$, the space and time for the counting array dominate, making it inefficient compared to other sorting algorithms like quicksort or merge sort.

239. In a modified version of Counting Sort for negative numbers, what must be done?

- a) **Shift the numbers to make them positive.**
- b) Ignore negative numbers.
- c) Use a separate array for negatives.
- d) The algorithm doesn't support negatives.

Explanation:

To handle negative numbers, you can shift the range (e.g., add the absolute value of the minimum) to make all values non-negative before applying Counting Sort.

240. What is the purpose of the cumulative sum in Counting Sort?

- a) To count the total elements.
- b) **To determine the correct positions for elements in the output.**
- c) To sort the counting array.
- d) To remove duplicates.

Explanation:

The cumulative sum adjusts the counting array to reflect the final positions, allowing elements to be placed in sorted order.

241. Counting Sort is a:

- a) Comparison-based sort.
- b) Non-comparison-based sort.**
- c) Hybrid sort.
- d) Recursive sort.

Explanation:

Counting Sort does not compare elements; it uses arithmetic operations based on keys, classifying it as a non-comparison-based sort.

242. For an array of 10 elements with values from 1 to 5, how many passes does Counting Sort typically make?

- a) 1 pass
- b) 2 passes
- c) 3 passes**
- d) Variable passes

Explanation:

Counting Sort generally involves three main passes: one to count frequencies, one to compute cumulative sums, and one to build the output array.

243. If the input array is [4, 2, 2, 8, 3, 3, 1], what will be the first element in the sorted output using Counting Sort?

- a) 1
- b) 2
- c) 3
- d) 4

Explanation:

After processing, the smallest element, 1, will be placed first in the output array based on the counting and cumulative arrays.

244. Counting Sort can be unstable if:

- a) Elements are not integers.
- b) The implementation does not preserve order of equal elements.**
- c) The array has duplicates.
- d) It is always stable.

Explanation:

Counting Sort can be implemented to be stable by processing elements in reverse order when placing them in the output array, but a poor implementation might make it unstable.

245. Compared to Bubble Sort, Counting Sort is better for:

- a) Nearly sorted arrays.
- b) Arrays with a small range of values.**
- c) Random arrays of large size.
- d) Arrays with floating-point numbers.

Explanation:

Bubble Sort is $O(n^2)$, while Counting Sort is $O(n + k)$ for small k , making it superior for arrays with limited distinct values.

246. In Counting Sort, the final output array is:

- a) The same as the input array.
- b) A new array with elements in sorted order.**
- c) Reversed from the input.
- d) Partially sorted.

Explanation:

The algorithm produces a completely sorted array as output, based on the frequencies and positions derived from the counting array.

247. Which of the following hash functions is most likely to cause clustering in a hash table? Here k is the input key value and m is hash table size. You may assume that all four hash functions generate valid indexes in the hash table.

a) $h(k) = k \% m$

b) $h(k) = \text{floor}(m * (k \bmod 1))$

c) $h(k) = k$

d) $h(k) = ((k / m) + k * m) + k \% m$

Explanation

The modulo operation in option a) results in clustering since the hash values of keys that are close to each other will also be close, leading to collisions. Options B, C, and D use more complex hash functions, which spread out the keys more uniformly across the table.

248. What are the methods to handle collision:

a) Separate Chaining

b) Open Chaining

c) Both of the above

d) None of the above

Explanation

There are mainly two methods to handle collision:

- Separate Chaining
- Open Addressing

249. What are the applications of linear probing including?

- a) Databases
- b) Caching
- c) Compiler Design
- d) All of the above**

Explanation

Linear probing is a collision handling technique used in hashing, where the algorithm looks for the next available slot in the hash table to store the collided key. Some of the applications of linear probing include:

- **Caching:** Linear probing can be used in caching systems to store frequently accessed data in memory. When a cache miss occurs, the data can be loaded into the cache using linear probing, and when a collision occurs, the next available slot in the cache can be used to store the data.
- **Databases:** Linear probing can be used in databases to store records and their associated keys. When a collision occurs, linear probing can be used to find the next available slot to store the record.
- **Compiler design:** Linear probing can be used in compiler design to implement symbol tables, error recovery mechanisms, and syntax analysis.

Hence Option(D) is the correct answer.

250. Which is a simple form of hashing where the data is directly mapped to an index in a hash table.

a) Collision

b) Index Mapping

c) Functional Mapping

d) Hash function

Explanation

Index Mapping (also known as Trivial Hashing) is a simple form of hashing where the data is directly mapped to an index in a hash table. The hash function used in this method is typically the identity function, which maps the input data to itself. In this case, the key of the data is used as the index in the hash table, and the value is stored at that index.

Hence Option(B) is the correct answer.

251. Suppose we are given n keys, m hash table slots, and two simple uniform hash functions h_1 and h_2 . Further, suppose our hashing scheme uses h_1 for the odd keys and h_2 for the even keys. What is the expected number of keys in a slot?

a) $n/2m$

b) n/m

c) m/n

d) $2n/m$

Explanation

For a uniform hash function, regardless of the number of hash functions, the anticipated number of keys in a slot is always computed as $(\text{number of keys})/(\text{number of slots})$.

252. Which of the following are components of Hashing?

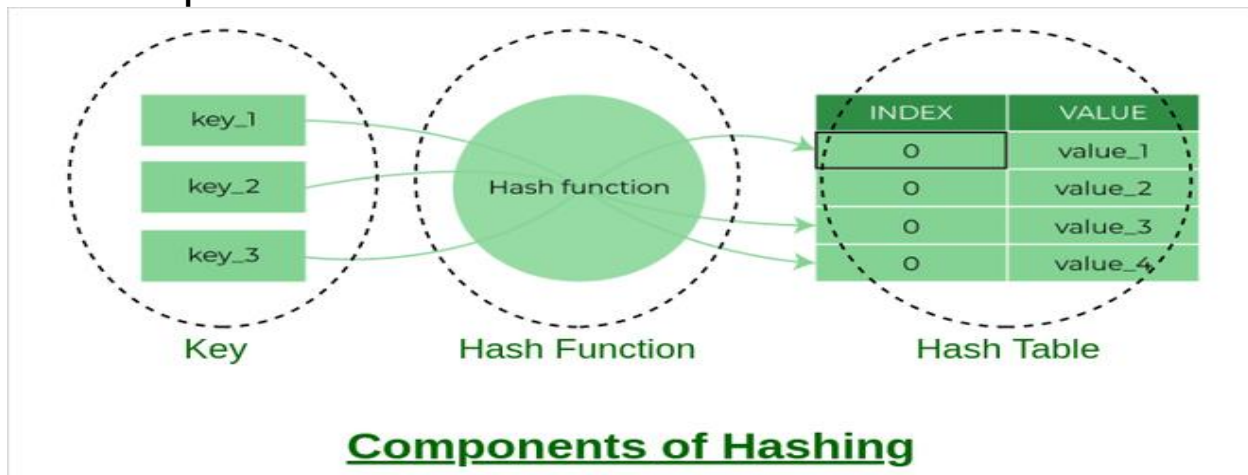
- a) Key
- b) Hash Function
- c) Hash Table
- d) **All of the above**

Explanation

Components of Hashing

There are majorly three components of hashing:

1. **Key:** A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. **Hash Function:** The hash function receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index.
3. **Hash Table:** Hash table is a data structure that maps keys to values using a special function called a hash function. Hash stores the data in an associative manner in an array where each data value has its own unique index.



253. A hash table of length 10 uses open addressing with hash function $h(k)=k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below.

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- a) 46, 42, 34, 52, 23, 33
- b) 34, 42, 23, 52, 33, 46
- c) 46, 34, 42, 23, 52, 33**
- d) 42, 46, 33, 23, 34, 52

Explanation

Here linear probing is used with the hash function $h(k)=k \bmod 10$.

Let's analyze all the options carefully:

Key A B C D

0

1

2 42 42 42 42

3 52 23 23 33

4 34 34 34 23

5 23 52 52 34

6 46 33 46 46

7 33 46 33 52

8

9

After analyzing all the options (C) is the correct possible order in which the key values could have been inserted in the table

(A) doesn't create the hash table as the element 52 appears before 23 in this sequence.

(B) doesn't create the hash table as the element 33 appears before 46 in this sequence.

(C) create the hash table as the element 42, 23 and 34 appear before 52 and 33, and 46 appears before 33.

(D) doesn't create the hash table as element 33 appears before 23 in this sequence.

254. The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and linear probing. What is the resultant hash table?

0	
1	
2	2
3	23
4	
5	15
6	
7	
8	18
9	

(A)

0	
1	
2	12
3	13
4	
5	5
6	
7	
8	18
9	

(B)

0	
1	
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15

(C)

0	
1	
2	12, 2
3	13, 3, 23
4	
5	5, 15
6	
7	
8	18
9	

(D)

- a) A
- b) B
- c) C**
- d) D

Explanation

Open addressing, or closed hashing, is a method of collision resolution in hash tables. With this method a hash collision is resolved by probing, or searching through alternate locations in the array (the probe sequence) until either the target record is found, or an unused array slot is found, which indicates that there is no such key in the table.

Well known probe sequences include: *linear probing* in which the interval between probes is fixed--often at 1.

quadratic probing in which the interval between probes increases linearly (hence, the indices are described by a quadratic function).

double hashing in which the interval between probes is fixed for each record but is computed by another hash function.

255. Consider a hash table of size seven, with starting index zero, and a hash function $(7x+3) \bmod 4$. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1, 3, 8, 10 is inserted into the table using closed hashing? Here “__” denotes an empty location in the table.

- a) 3, 10, 1, 8, __, __, __
- b) 1, 3, 8, 10, __, __, __
- c) 1, __, 3, __, 8, __, 10
- d) 3, 10, __, __, 8, __, __

Explanation

Keys are 1, 3, 8, 10. $h(x) = (7x + 3)$

$$h(1) = (7 \cdot 1 + 3) \bmod 4 = 10 \bmod 4 = 2$$

$$h(3) = (7 \cdot 3 + 3) \bmod 4 = 24 \bmod 4 = 0$$

$$h(8) = (7 \cdot 8 + 3) \bmod 4 = 59 \bmod 4 = 3$$

$$h(10) = (7 \cdot 10 + 3) \bmod 4 = 73 \bmod 4 = 1$$

i.e. 3, 10, 1, 8, __, __, __

256. Given a hash table T with 25 slots that stores 2000 elements, the load factor α for T is _____

- a) 80
- b) 0.0125
- c) 8000
- d) 1.25

Explanation

load factor = (no. of elements) / (no. of table slots) = $2000/25 = 80$.

257. An advantage of chained hash table (external hashing) over the open addressing scheme is

- a) Worst case complexity of search operations is less
- b) Space used is less
- c) Deletion is easier**
- d) None of the above

Explanation

In Open Addressing scheme sometimes though element is present we can't delete it if empty bucket comes in between while searching for that element. External hashing scheme is free from this limitation.

Hence, Option C is the correct answer.

258. What will be the cipher text produced by the following cipher function for the plain text ISRO with key $k = 7$.

[Consider 'A' = 0, 'B' = 1, 'Z' = 25].

$$Ck(M) = (kM + 13) \bmod 26.$$

- a) RJCH**
- b) QIBG
- c) GQPM
- d) XPIN

Explanation

$$Ck(M) = (kM + 13) \bmod 26$$

Here, 'A' = 0, 'B' = 1, 'Z' = 25

$$I = Ck(I) = (7 \cdot 8 + 13) \bmod 26 = 17 = R$$

$$S = Ck(S) = (7 \cdot 18 + 13) \bmod 26 = 9 = J$$

$$R = Ck(R) = (7 \cdot 17 + 13) \bmod 26 = 2 = C$$

$$O = Ck(O) = (7 \cdot 14 + 13) \bmod 26 = 7 = H$$

So, option (A) is correct.

259. A hash function h defined $h(\text{key}) = \text{key} \bmod 7$, with linear probing, is used to insert the keys 44, 45, 79, 55, 91, 18, 63 into a table indexed from 0 to 6. What will be the location of key 18?

- a) 3
- b) 4
- c) 5**
- d) 6

Explanation

keys 44, 45, 79, 55, 91, 18, 63

$h(\text{key}) = \text{key} \bmod 7$

$h(44) = 44 \bmod 7 = 2$

$h(45) = 45 \bmod 7 = 3$

$h(79) = 79 \bmod 7 = 2$ but 2 is already filled 44, linear probing is applied but 3 is also filled.

So, 79 will occupy 4.

$h(55) = 55 \bmod 7 = 6$

$h(91) = 91 \bmod 7 = 0$

$h(18) = 18 \bmod 7 = 4$ but 4 is occupied by 79 so, it will occupy 5.

$h(63) = 63 \bmod 7 = 0$. 0 is also occupied so, it will occupy 1.

So, option (C) is correct.

260. A hash table with 10 buckets with one slot per bucket is depicted here. The symbols, S1 to S7 are initially entered using a hashing function with linear probing. The maximum number of comparisons needed in searching an item that is not present is

- a) 4
- b) 5**
- c) 6
- d) 3

Explanation

We have to consider different cases to find the maximum number of comparisons needed in searching an item that is not present.

If searching starts at 0 index: 3 comparisons will be made at index 0, index 1 and finally at index 2 and terminate searching when index 2 found empty. Similarly, after checking on different indexes, the maximum number of comparisons will be required if searching starts from index 8, as in this case, total 5 comparisons will be made at index 8, 9, 0, 1 and 2. So, correct option is (B).

261. Consider a hash table of size $m = 100$ and the hash function $h(k) = \text{floor}(m(kA \bmod 1))$ for

$$A = \text{frac}(\sqrt{5} - 1)/2 = 0.618033$$

Compute the location to which the key $k = 123456$ is placed in hash table.

a) 77

b) 82

c) 88

d) 89

Explanation

$$h(k) = \text{floor}(m(kA \bmod 1))$$

$$M = 100;$$

$$k = 123456$$

$$h(k) = \text{floor}(100(123456 * 0.618033 \bmod 1))$$

$$h(k) = \text{floor}(100(76189.882048 \bmod 1))$$

$$h(k) = \text{floor}(100 * 0.882048)$$

$$h(k) = \text{floor}(88.2048)$$

$$h(k) = 88.$$

So, option (C) is correct.

262. Consider a 13-element hash table for which $f(\text{key}) = \text{key} \bmod 13$ is used with integer keys. Assuming linear probing is used for collision resolution, at which location would the key 103 be inserted, if the keys 661, 182, 24 and 103 are inserted in that order?

- a) 0
- b) 1**
- c) 11
- d) 12

Explanation

$661 \bmod 13 = 11$

$182 \bmod 13 = 0$

$24 \bmod 13 = 11$, already filled, so after linear probing it will get index 12

$103 \bmod 13 = 12$, already filled, so after linear probing it will get index 1

182	103										661	24
0	1	2	3	4	5	6	7	8	9	10	11	12

263. Which searching technique takes $O(1)$ time complexity for searching the data?

- a) Binary Search
- b) Linear Search
- c) AVL Tree Search
- d) Hashing**

Explanation

Hashing is the technique that takes $O(1)$ time complexity for searching the data.

264. Consider a hash table of size $m = 10000$, and the hash function $h(K) = \text{floor}(m(KA \bmod 1))$ for $A = (\sqrt{5} - 1)/2$. The key 123456 is mapped to location _____.

- a) 46
- b) 41**
- c) 43
- d) 48

Explanation

Given hash function: $h(K) = \text{floor}(m(KA \bmod 1))$ where $A = (\sqrt{5} - 1)/2$

$$\begin{aligned} h(123456) &= \text{floor}(10000 * (123456 * (\sqrt{5} - 1) / 2) \bmod 1) \\ &= \text{floor}(10000 * (76300.004115 \bmod 1)) \\ &= \text{floor}(10000 * (.004115)) \\ &= 41.15 \\ &= 41 \end{aligned}$$

So, option (B) is correct.

265. Insert the characters of the string **K R P C S N Y T J M** into a hash table of size 10. Use the hash function

$$h(x) = (\text{ord}(x) - \text{ord}("A") + 1) \bmod 10$$

If linear probing is used to resolve collisions, then the following insertion causes collision

- a) Y
- b) C
- c) M**
- d) P

Explanation

(a) The hash table with size 10 will have index from 0 to 9.

hash function = $h(x) = ((\text{ord}(x) - \text{ord}(A) + 1)) \bmod 10$, So for string K R P C S N Y T J M:

K will be inserted at index : $(11-1+1) \bmod 10 = 1$

R at index: $(18-1+1) \bmod 10 = 8$

P at index: $(16-1+1) \bmod 10 = 6$

C at index: $(3-1+1) \bmod 10 = 3$

S at index: $(19-1+1) \bmod 10 = 9$

N at index: $(14-1+1) \bmod 10 = 4$

Y at index $(25-1+1) \bmod 10 = 5$

T at index $(20-1+1) \bmod 10 = 0$

J at index $(10-1+1) \bmod 10 = 0$ // first collision occurs.

M at index $(13-1+1) \bmod 10 = 3$ //second collision occurs. Only J and M are causing the collision.

(b) Final Hash table will be:

0	T
1	K
2	J
3	C
4	N
5	Y
6	P
7	M
8	R
9	S

266. Which of the following statement(s) is TRUE?

- I. A hash function takes a message of arbitrary length and generates a fixed length code.
- II. A hash function takes a message of fixed length and generates a code of variable length.
- III. A hash function may give the same hash value for distinct messages.

a) I only

b) II and III only

c) I and III only

d) II only

Explanation

Hash function is defined as any function that can be used to map data of arbitrary size of data to a fixed size data. The values returned by a hash function are called hash values, hash codes, digests, or simply hashes.

Statement 1 is correct; it is possible that a Hash Function maps a value to a same location in the memory that's why collision occurs and we have different technique to handle this problem.

Statement 3 is correct. eg: we have hash function,

$$h(x) = x \bmod 3$$

Acc to Statement 1, no matter what the value of 'x' is h(x) results in a fixed mapping location. Acc. to Statement 3, h(x) can result in same mapping location for different value of 'x' e.g. if $x = 4$ or $x = 7$, $h(x) = 1$ in both the cases, although collision occurs.

1. What is the time complexity of the following code?

```
def fun(n,m):  
    for i in range(n):  
        print(i)  
    for i in range(m):  
        print(i)
```

Answer

Time complexity: $O(n+m)$

Here the first loop runs for n times and the second loop iterates for m times.

Thus time complexity is $O(n+m)$.

2. What is the time complexity of the following code?

```
import random  
def fun(N):  
    counter=0  
    for i in range(N):  
        counter+=random.randint(1,100)  
    print(counter)
```

Answer

Time complexity: $O(N)$

Here the loop is running for N times.

The random.randint(a,b) function takes $O(1)$ time.

Thus the time complexity is $O(N)$.

3. What is the time and space complexity of the following code?

```
def fun(N,M):  
    arr=[ ]  
    counter=0  
    for i in range(N):  
        arr.append(i)  
    for i in range(M):  
        counter+=1  
    print(counter)
```

Answer

Time complexity: $O(N+M)$, Space complexity: $O(N)$.

Here the first loop runs for N times. So the array “arr” will have a size of $O(N)$.

Also, the second loop runs for M iterations.

Thus time complexity is $O(N) + O(M) = O(N+M)$.

Since we make an array of size $O(N)$, the space complexity is $O(N)$.

4. What is the time complexity of the following code?

```
def function(N,M):  
    counter=0  
    for i in range(N):  
        for j in range(M):  
            counter+=1  
    print(counter)
```

Answer

Time complexity: $O(N \times M)$.

Here the outer loop runs for N times. In the i^{th} iteration, the inner loop runs for M times and hence the time complexity is $O(M)$. As the loop runs for N times, the time complexity is $N \times M$ i.e. $O(N \times M)$

5. What is the time complexity of the code given below?

```
#include<bits/stdc++.h>
using namespace std;

int i, j, k = 0;
while(i<=n){
    j=2;
    while(j<=n){
        k = k + n / 2;
        j = j * 2;
    }
    i++;
}
```

Answer

Time complexity: $O(n * \log(n))$

If we look closely, we can see that 'j' keeps doubling until it's less than or equal to 'n'.

The number of times which we can keep doubling a number until it's less than or equal to 'n' is $\log(n)$.

Let's look at a few examples.

'j' = 2, 4, 8, 16 for n = 16

Steps taken = $\log(16) = 4$.

As a result, j will run for $O(\log n)$ steps.

For 'i', it takes n/2 iterations costing us $O(n/2)$ time.

As a result, total time complexity is equal $O(n/2 * \log(n)) \sim O(n * \log(n))$.

6. What is the time complexity of the code snippet given below?

```
def f():  
    ans = 0  
    for i = 1 to n:  
        for j = i; j <= n; j += i:  
            ans += 1  
    print(ans)
```

Answer

Time complexity: $O(n * \log(n))$

Here outer for loop (loop i) will run 'n' times.

when $i = 1$, inner loop (loop j) will run $n/1$ times [for $j = 1; j \leq n; j += 1$]

when $i = 2$, inner loop will run $n/2$ times [for $j = 2; j \leq n; j += 2$]

...up to $i = n$, inner loop will run $n/n = 1$ time [for $j = n; j \leq n; j += n$]

So, Total time = $O(n + n/2 + n/3 + \dots + n/n)$

$= O(n(1 + 1/2 + 1/3 + \dots + 1/n))$

$= O(n * \log n)$

7. Provide big-O timing estimates for the following algorithm:

```
function Max(a: ArrayOfIntegers) : integer;
```

{Finds the largest element in an array, a , of n elements}

```
var largest, i : integer;
```

begin

```
largest := 0;
```

for $i := 1$ to n do

if $a[i] > \text{largest}$ then

```
largest := a[i];
```

Max := latest

end;

Solution

1) **for** i:= 1 to n **do**

2) **if** $a[i] > \text{largest}$ **then** $O(1)$

3) largest := a[i]; $O(1)$

- The cost of statement (3) is $O(1)$
- The cost of the if statement (2) is $O(1)$
- The loop (2 – 3) is executed n times, with a cost of $O(n)$

8. Assignment Problem: Example

- You work as a sales manager for a toy manufacture, and you currently have three salespeople on the road meeting buyers. Your salespeople are in Austin, TX; Boston, MA; and Chicago, IL. You want them to fly to three other cities: Denver, CO; Edmonton, Alberta; and Fargo, ND.
- The table below shows the cost of airplane tickets in dollars between these cities.

From \ To	Denver	Edmonton	Fargo
Austin	250	400	350
Boston	400	600	350
Chicago	200	400	250

- Where should you send each of your salespeople in order to **minimize** airfare?

Solution

	D	E	F
A	250	400	350
B	400	600	350
C	200	400	250

<D, E, F>	cost = 250 + 600 + 250 = 1100
<D, F, E>	cost = 250 + 350 + 400 = 1000
<E, D, F>	cost = 400 + 400 + 250 = 1050
<E, F, D>	cost = 400 + 350 + 200 = 950
<F, D, E>	cost = 350 + 400 + 400 = 1150
<F, E, D>	cost = 350 + 600 + 200 = 1150

Thus, your salesperson should travel Austin to Edmonton, Boston to Fargo and Chicago TO Denver.

- The general case of the assignment problem is $n!$

9. What is the main difference between DFS and BFS?

Answer:

DFS explores as deep as possible along one branch before backtracking, while BFS explores all neighbors at the current level before moving to the next level.

10. When should I use DFS instead of BFS?

Answer:

Use DFS when you need to explore all possible paths, solve backtracking problems, or find connected components.

DFS is also useful for detecting cycles and performing topological sorting.

11. What are the time complexities of DFS and BFS?

Answer:

Both DFS and BFS have a time complexity of $O(V + E)$, where V is the number of vertices and E is the number of edges in the graph.

12. What are the space complexities of DFS and BFS?

Answer:

DFS has a space complexity of $O(V)$ due to the recursion stack (or explicit stack), while BFS also has a space complexity of $O(V)$, as it uses a queue to store nodes at the current level.

13. Can DFS be used to find the shortest path?

Answer:

DFS can find paths, but it is not guaranteed to find the shortest path in an unweighted graph. BFS should be used for shortest path problems.

14. Is BFS or DFS better for tree traversal?

Answer:

DFS is used for pre-order, in-order, and traversals in trees, while BFS is used for level-order traversal.

15. What is the difference between a binary tree and a binary search tree (BST)?

Answer

A binary tree is a general structure where each node can have up to two children. A binary search tree (BST) is a specific type of binary tree where each node's left child contains a value less than the parent node, and the right child contains a value greater than the parent node, allowing for efficient searching, insertion, and deletion.

16. What is the height of a binary tree?

Answer

The height of a binary tree is the number of edges on the longest path from the root node to a leaf node. A tree with a single node has a height of zero.

17. What is a leaf node in a binary tree?

Answer

A leaf node is a node that does not have any children. It is the end point of a branch in the tree.

18. What is the root of a binary tree?

Answer

The root is the topmost node of the binary tree, from which all other nodes descend. It is the starting point of the tree structure.

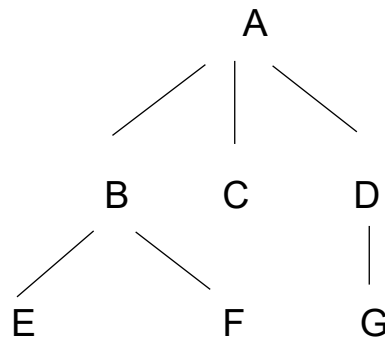
19. What is the difference between a full binary tree and a complete binary tree?

Answer

- **Full Binary Tree: Every node has either 0 or 2 children.**
- **Complete Binary Tree: All levels are fully filled except possibly the last level, which is filled from left to right.**

20. Consider the following graph and perform BFS and DFS starting from node A. The goal is to reach node G. Show the step-by-step execution and final output.

- Goal State: Reach node G.
- Requirements:
 - Show BFS and DFS Search order.
 - Track queue (BFS) and stack (DFS) at each step



Solution

BFS Search:

- BFS explores level by level using a queue (FIFO).

Step-by-step BFS execution (starting from A):

1. Visit A → Queue: [B, C, D]
2. Visit B → Queue: [C, D, E, F]
3. Visit C → Queue: [D, E, F]
4. Visit D → Queue: [E, F, G]
5. Visit E → Queue: [F, G]
6. Visit F → Queue: [G]
7. Visit G → Goal Reached

Final BFS Output: A → B → C → D → E → F → G

DFS Search:

- **DFS explores one branch fully before backtracking, using a stack (LIFO).**

Step-by-step DFS execution (starting from A):

- 1. Visit A → Stack: [D, C, B]**
- 2. Visit B → Stack: [D, C, F, E]**
- 3. Visit E → Stack: [D, C, F]**
- 4. Visit F → Stack: [D, C]**
- 5. Visit C → Stack: [D]**
- 6. Visit D → Stack: [G]**
- 7. Visit G → Goal Reached**

Final BFS Output: A → B → E → F → C → D → G

21. Consider the following tree

If the **post order traversal** gives **ab-cd*+** then the label of the nodes 1,2,3,... will be:

Answer

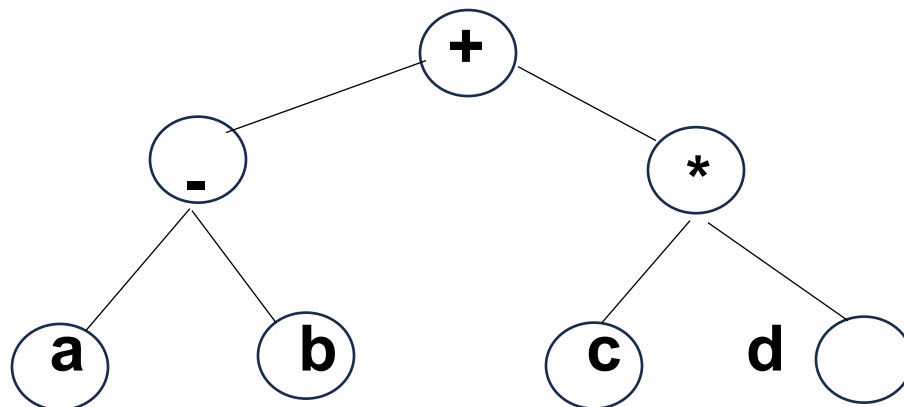
Post order traversal of the given binary tree will give the following sequence: 4 5 2 6 7 3 1.

Now comparing the sequence with a b - c d * + we get:

1 = +,
2 = -,
3 = *,
4 = a,
5 = b,
6 = c,
and 7 = d.

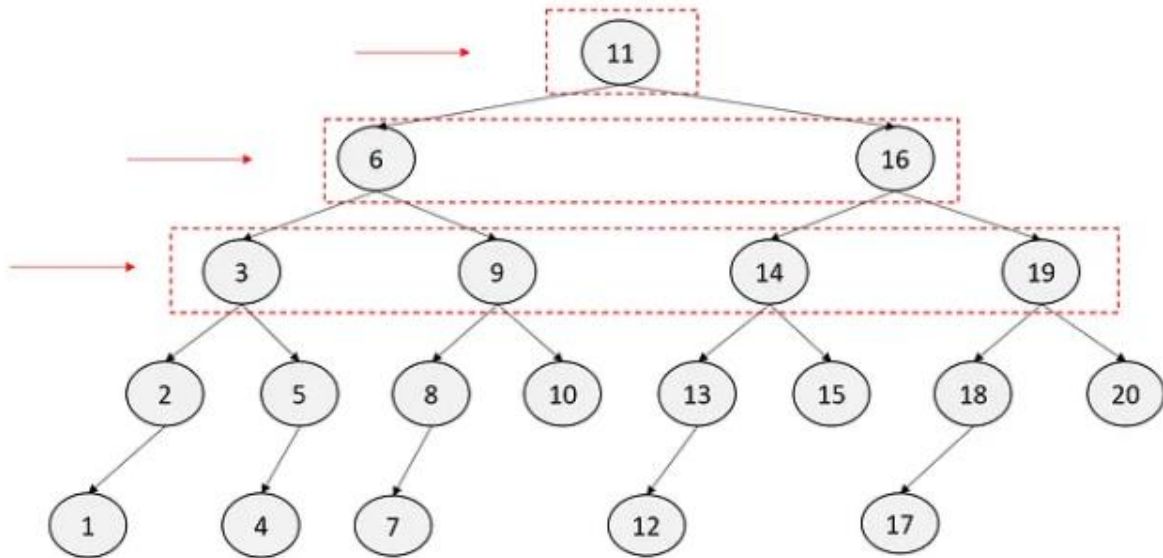
so, the sequence is:

+,-,*,a,b,c,d



22. Example:

BFS



The values of the queue for different iteration of the while loop is as below.

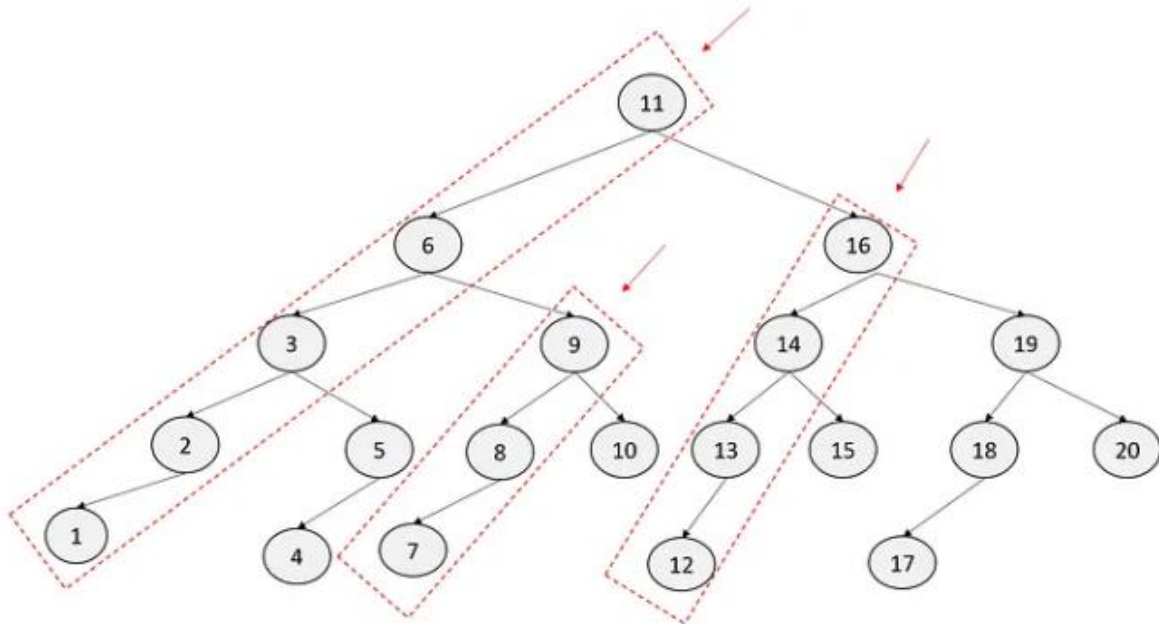
```
11          #start
6, 16       #removed 11 and added its connected.
3, 9, 14, 19 #removed 6 and 16 added their connected nodes
2, 5, 8, 10, 13, 15, 18, 20
```

Result:

```
11 -> 6 -> 16 -> 3 -> 9 -> 14 -> 19 -> 2 -> 5 -> 8 -> 10 -> 13 -> 15
-> 18 -> 20 -> 1 -> 4 -> 7 -> 12 -> 17
```

23. Example

DFS



The values of the stack in diff iteration inside while is as follows:

11	6	3	2	1
11	6	9	5	1
11	6	9	9	5
11	16	16	16	16

Result:

```
11 -> 6 -> 3 -> 2 -> 1 -> 5 -> 4 -> 9 -> 8 -> 7 -> 10 -> 16 -> 14 ->
13 -> 12 -> 15 -> 19 -> 18 -> 17 -> 20
```

24.

BFS and DFS questions

Question 1: Consider the following graph and perform BFS and DFS starting from node A. The goal is to reach node G. Show the step-by-step execution and final output.



- **Goal State: Reach node G.**
- **Requirements:**
 - o Show BFS and DFS Search order.
 - o Track queue (BFS) and stack (DFS) at each step.
 - o Explain the difference in the paths found by both algorithms.

Answer: BFS and DFS Search for Given Graph

Graph:



Goal State: Reach node G

Breadth-First Search (BFS) Search:

- BFS explores level by level using a queue (FIFO).

Step-by-step BFS execution (starting from A):

1. **Visit A** → Queue: [B, C, D]
2. **Visit B** → Queue: [C, D, E, F]
3. **Visit C** → Queue: [D, E, F]
4. **Visit D** → Queue: [E, F, G]
5. **Visit E** → Queue: [F, G]

6. **Visit F** → Queue: [G]

7. **Visit G** → **Goal Reached**

Final BFS Output: A → B → C → D → E → F → G

Depth-First Search (DFS) Search:

- DFS explores one branch fully before backtracking, using a stack (LIFO).

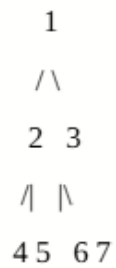
Step-by-step DFS execution (starting from A):

1. **Visit A** → Stack: [D, C, B]
2. **Visit B** → Stack: [D, C, F, E]
3. **Visit E** → Stack: [D, C, F]
4. **Visit F** → Stack: [D, C]
5. **Visit C** → Stack: [D]
6. **Visit D** → Stack: [G]
7. **Visit G** → **Goal Reached**

Final DFS Output: A → B → E → F → C → D → G

25.

Question 2: For the given tree, perform BFS and DFS starting from node 1 to find the shortest path to node 7.



- **Goal State:** Reach node 7.

- **Requirements:**
 - o Show step-by-step execution of both algorithms.
 - o Explain which algorithm finds the shortest path.

Answer: BFS and DFS Search for Tree

Tree:

```
    1
   /\
  2 3
 /\  \
4 5 6 7
```

Goal State: Reach node 7

Breadth-First Search (BFS) Search:

- BFS explores level by level.

Step-by-step BFS execution (starting from 1):

1. **Visit 1** → Queue: [2, 3]
2. **Visit 2** → Queue: [3, 4, 5]
3. **Visit 3** → Queue: [4, 5, 6, 7]
4. **Visit 4** → Queue: [5, 6, 7]
5. **Visit 5** → Queue: [6, 7]
6. **Visit 6** → Queue: [7]
7. **Visit 7** → **Goal Reached**

Final BFS Output: 1 → 2 → 3 → 4 → 5 → 6 → 7

Depth-First Search (DFS) Search:

- DFS explores one branch fully before backtracking.

Step-by-step DFS execution (starting from 1):

1. **Visit 1** → Stack: [3, 2]
2. **Visit 2** → Stack: [3, 5, 4]
3. **Visit 4** → Stack: [3, 5]

4. **Visit 5** → Stack: [3]
5. **Visit 3** → Stack: [7, 6]
6. **Visit 6** → Stack: [7]
7. **Visit 7** → **Goal Reached**

Final DFS Output: 1 → 2 → 4 → 5 → 3 → 6 → 7

Explain which algorithm finds the shortest path.

- **BFS** always finds the shortest path in an **unweighted graph** because it explores all nodes at each level before moving deeper.
 - **DFS** does not guarantee the shortest path since it follows one path deeply before backtracking.
 - In this case, **BFS finds the shortest path (1 → 3 → 7)**, while DFS might take a longer path (1 → 2 → 4 → 5 → 3 → 6 → 7).
-

Analysis Methods

There are different analysis methods. The most common are given below.

Asymptotic Analysis

In asymptotic analysis, there are three main asymptotic notations.

- **Big O Notation:** This notation represents the worst case. It only calculates the upper bound of time when the algorithm is implemented
- **Omega Notation:** This notation represents the best case. It only calculates the lower bound of time when the algorithm is implemented
- **Theta Notation:** It is used for analyzing the average case. It calculates the time complexity using the upper and lower bounds

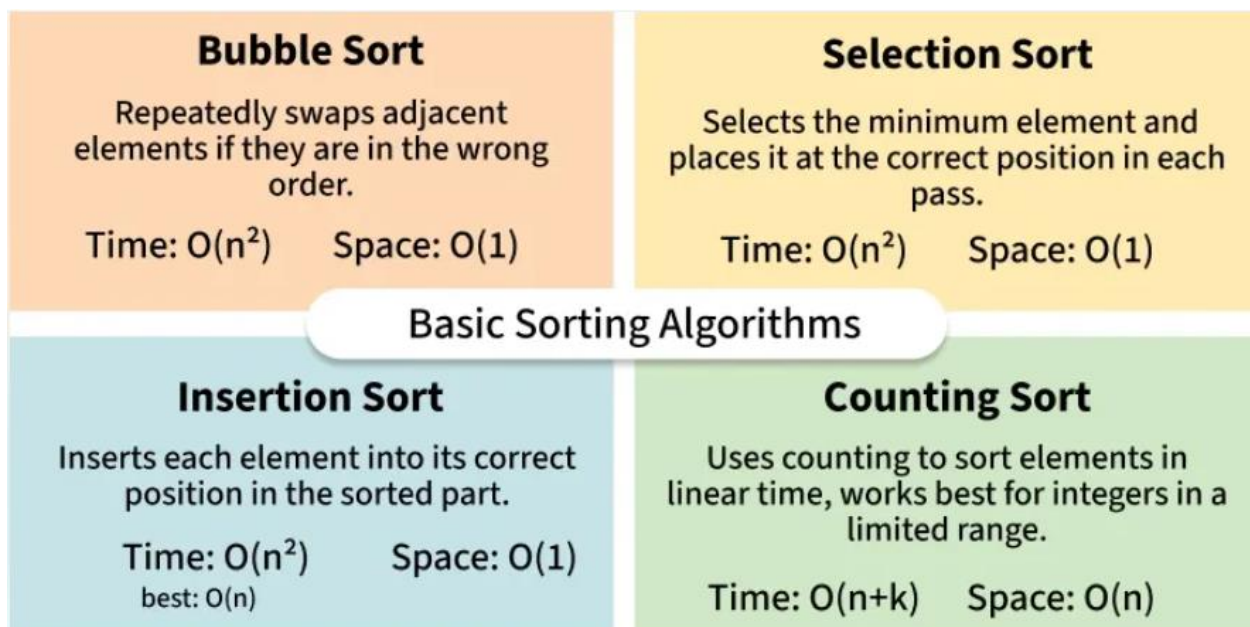
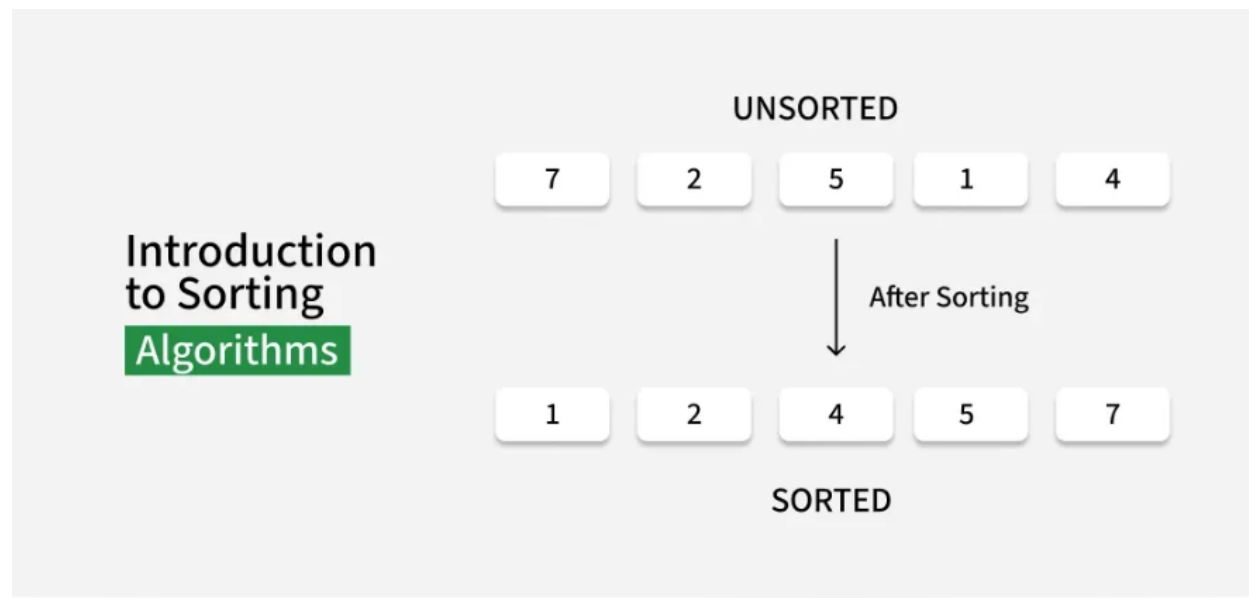
Quick Overview of Sorting Algorithms

Algorithm	Average Case Complexity	Worst Case Complexity	Space Complexity	Stability
Bubble Sort	$O(n^2)$	$O(n^2)$	Constant	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	Constant	No
Insertion Sort	$O(n^2)$	$O(n^2)$	Constant	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	In-Place*	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	In-Place	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes

Categories Overview

Algorithm	Method	In-Place	Adaptive	Stability	Swaps / Inversions	Recursion vs. Iteration
QuickSort	Divide and Conquer	Yes*	No*	Unstable	Few	Recursive
Bubble Sort	Exchange	Yes	Yes	Stable	Multiple	Iterative
Merge Sort	Divide and Conquer	No	No	Stable	-	Recursive
Counting Sort	Non-comparative	No	No	Stable	-	Iterative
Insertion Sort	Incremental	Yes	Yes	Stable	Few	Iterative
HeapSort	Selection	Yes	No	Unstable	-	Iterative

Sorting refers to rearrangement of a given array or list of elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of elements in the respective data structure.



Merge Sort

A divide-and-conquer algorithm that splits and merges arrays in sorted order.

Time: $O(n \cdot \log n)$ Space: $O(n)$

Quick Sort

Divides the array using a pivot and recursively sorts the partitions.

Time: $O(n^2)$ Space: $O(\log(n))$
Avg: $O(n \log n)$

Advanced Sorting Algorithms

Heap Sort

Builds a heap and repeatedly extracts the maximum (or minimum) to sort the array.

Time: $O(n \cdot \log n)$ Space: $O(1)$

Counting Sort is a linear-time sorting algorithm designed for sorting arrays of integers where the range of input values is not significantly larger than the number of elements to be sorted. Unlike comparison-based sorts, it uses the actual values of the elements to determine their positions in the output array.

The algorithm works by first determining the range of the input elements, typically by finding the maximum value. It then creates a count array to store the frequency of each distinct element in the input. After populating the count array, it is modified to reflect the cumulative counts, which helps in placing elements in their correct sorted positions.

For example, given an array [4, 2, 2, 8, 3, 3, 1]:

- The count array might look like [0, 1, 2, 2, 1, 0, 0, 0, 1] for values up to 8.**
- Using the cumulative counts, the sorted array becomes [1, 2, 2, 3, 3, 4, 8].**

This method is stable and efficient, with a time complexity of $O(n + k)$, where n is the number of elements and k is the range of input values. It excels in scenarios with non-negative integers and a limited range, but it is less suitable for floating-point numbers or large ranges due to high space requirements.

What is brute force?	Solving a problem directly but is often very inefficient. Can be used on a variety of problems.
What is selection sort?	A brute force sorting algorithm that scans array to find smallest element to swap with first element over a series of $n-1$ passes until its sorted. $O(n^2)$
What is a bubble sort?	A brute force sorting algorithm that compares adjacent elements and swapping them up which pushes large element up until $n-1$ passes. $O(n^2)$
What is sequential search?	A brute force search algorithm that searches each element in list until it finds a match to a key.
What is string matching?	A brute force algorithm that searches for a pattern of characters within a text. Worst Case $(n-m+1)$
Brute Force strengths and weaknesses?	Strengths: universal, simple, reasonable algorithms Weakness: Inefficient, slow, unconstructive.
What is an exhaustive search?	Problems that require you to search through all possible combinations to find the best one
What is a depth first search?	Search that starts at a specified vertex and explores as far as possible along each branch before back tracking and continuing the search until there are no more paths. It will then go to an unvisited vertex and start again.
What are properties of a depth first search?	Uses a stack, two orderings (along tree and back edges), used to check connectivity, acyclicity, and articulation points, Runtime for Adj matrix $O(V^2)$. Runtime for adj list is $O(V+E)$.
What is a Breadth first search?	Search that starts at a specified vertex and explores the Neighbours nodes first. Before moving to the next level of Neighbours.
What are properties of a breadth first search?	Uses a queue, one ordering, tree and cross edges, used to check connectivity, acyclicity, and min-edge paths, Runtime for Adj matrix $O(V^2)$. Runtime for adj list is $O(V+E)$.

Key Points

- **In-order Traversal:** Visits the left subtree first, then the root, and finally the right subtree.
- **Pre-order Traversal:** Visits the root first, then the left subtree, and finally the right subtree.
- **Post-order Traversal:** Visits the left subtree first, then the right subtree, and finally the root.
- **Commonality:** In all three traversal types, the left subtree is always visited before the right subtree.

Additional Information

- Traversal methods (in-order, pre-order, and post-order) are used to explore and process all nodes in a binary tree.
 - These traversal techniques differ in the order in which they visit the root node relative to the left and right subtrees.
 - Regardless of the specific traversal type, the left subtree is always visited before the right subtree, which is a key commonality
-
- **Divide and Conquer** is a fundamental algorithmic technique where a problem is broken down into smaller sub-problems, each of which is solved independently, and then the solutions to the sub-problems are combined to form the solution to the original problem.
 - **Binary Search**, **Merge Sort**, and **Quick Sort** are all classic examples of divide and conquer algorithms:
 - **Binary Search:** This algorithm works by dividing the search interval in half repeatedly until the target value is found or the interval is empty.
 - **Merge Sort:** This sorting algorithm divides the array into two halves, recursively sorts each half, and then merges the sorted halves to produce the final sorted array.
 - **Quick Sort:** This sorting algorithm selects a 'pivot' element and partitions the array into two sub-arrays according to whether elements are less than or greater than the pivot, then recursively sorts the sub-arrays.
 - **Heap Sort**, however, is not a divide and conquer algorithm. It is an efficient, comparison-based sorting algorithm that builds a heap data structure from the input data and then repeatedly extracts the maximum element from the heap to build the sorted array.